Инструкция по работе в интегрированной среде разработки AMD(Xilinx) Vivado

Оглавление

Обозначения и сокращения 3

Введение 4

Работа со средой разработки AMD(Xilinx) Vivado 4

1. Создание нового проекта и открытие уже существующего 4

2. Описание графического интерфейса среды Vivado 9

3. Работа с файлами проекта 11

4. Работа с IP — ядрами, доступными в среде Vivado 16

4.1. Clocking Wizard 16

4.2. Block Memory Generator 24

5. RTL анализ проекта 32

6. Синтез 34

7. Имплементация 37

8. Генерация битстрима и загрузка конфигурации в ПЛИС 41

Список использованных источников и литературы 42

ОБОЗНАЧЕНИЯ И СОКРАЩЕНИЯ

ПЛИС – программируемая логическая интегральная схема;

IP-ядра – готовые блоки, специфицированные на языке описания аппаратуры (IP – от англ. intellectual property);

GUI – (от англ. graphical user interface)графический пользовательский интерфейс ;

RTL – (от англ. Register transfer level) уровень регистровых передач;

VCO – (от англ. Voltage controlled oscillator) генератор управляемый напряжением;

BRAM – (от англ. Block Random Access Memory) блочная память с произвольным доступом;

TCL – (от англ. Tool Command Language — командный язык инструментов) — скриптовый язык высокого уровня;

PLL – (от англ. Phase Locked Loop) фазовая автоподстройка частоты — система автоматического регулирования, подстраивающая фазу управляемого генератора так, чтобы она была равна фазе опорного сигнала либо отличалась на известную функцию от времени;

MMCM – (от англ. Mixed Mode Clock Manager) конфигурируемый блок, который включает в себя систему ФАПЧ и фазовращатели для обеспечения точного управления тактовой частотой в ПЛИС Xilinx;

AXI – (от англ. Advanced eXtensible Interface) открытый стандарт интерфейсов, используемый многими вендорами IP.

Введение

В данной инструкции приведено описание базовой работы в среде разработки AMD(Xilinx) Vivado в GUI — режиме. Описан маршрут от создания проекта до получения битстрим — файла для загрузки конфигурации в ПЛИС на вашем устройстве.

Работа со средой разработки AMD(Xilinx) Vivado

1. Создание нового проекта и открытие уже существующего

Для запуска Vivado и создания проекта исполните команду `make deploy`, скрипт автоматически создаст проект, добавит в него файлы демонстрационного проекта и откроет GUI.

Если вы уже создали проект, но, например, закрыли Vivado, введите в терминале команду `vivado` и откроется графический интерфейс как на рисунке 1.

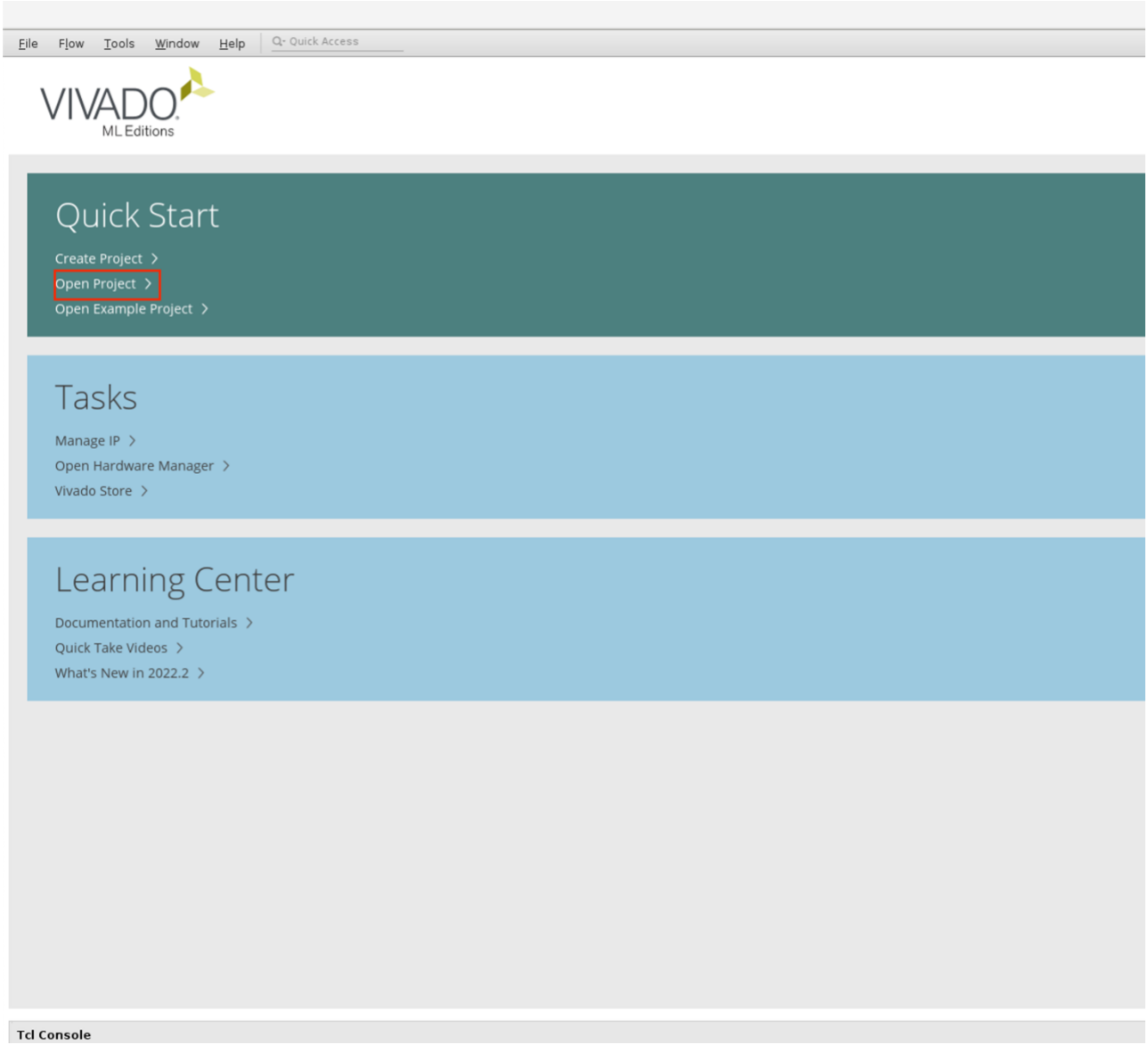


Рисунок 1 — Графический интерфейс стартового окна AMD(Xilinx) Vivado

Нажмите на кнопку *«Open project»*, далее в открывшемся окне найдите директорию вашего проекта, найдите в ней файл с расширением **.xpr* как показано на рисунке 2 и откройте его.

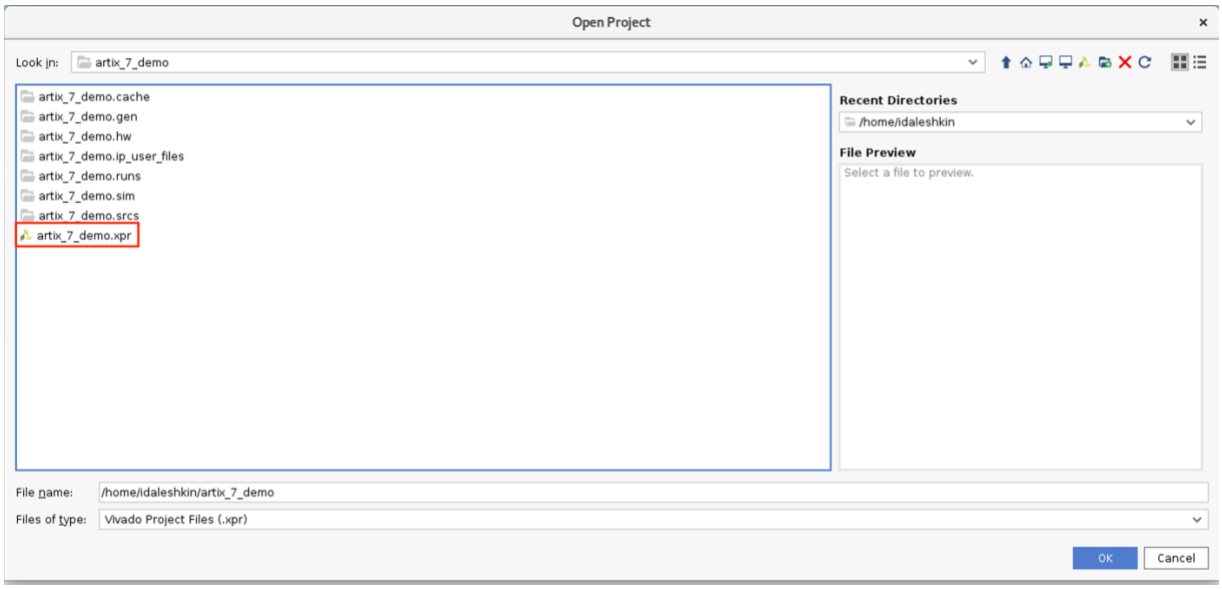


Рисунок 2 — Файл проекта *.xpr

В случае, если версии используемой среды и среды, в которой создавался проект различаются, будет предложено автоматически обновить проект до используемой версии либо открыть в режиме чтения, как показано на рисунке 3.

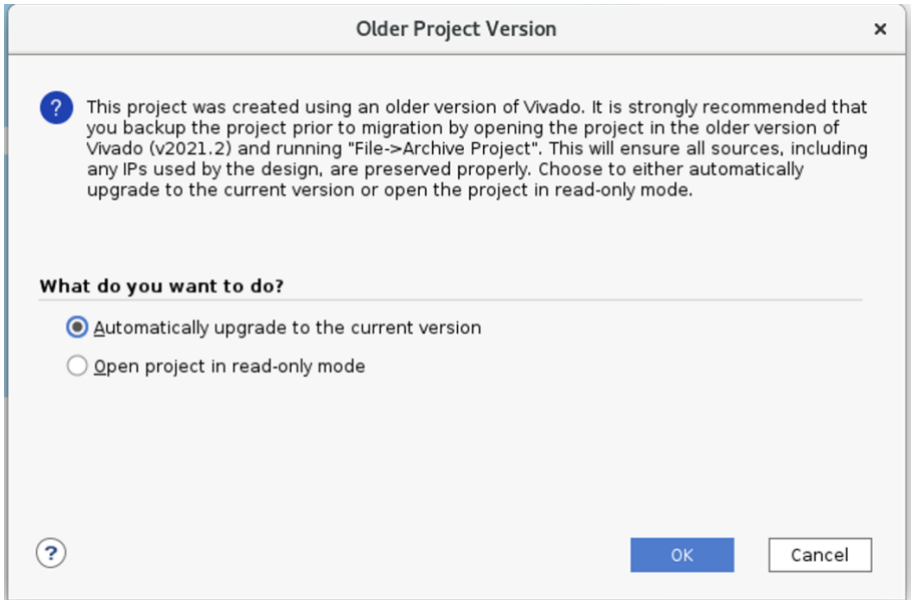


Рисунок 3 — Обновление проекта до нынешней версии среды

После обновления проекта среда сообщит, если в проекте есть IP-ядра старых версий для того, чтобы обновить их до актуальных нужно нажать *«Report IP Status»* как показано на рисунке 4.

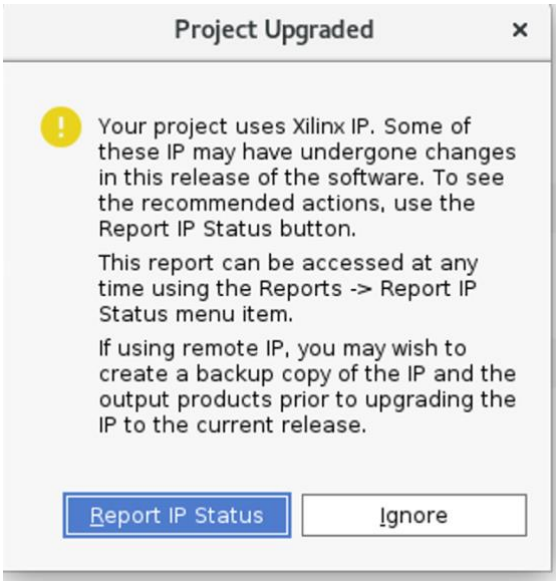


Рисунок 4 — Сообщение о том, что проект обновлен

Внизу интерфейса Vivado откроется отчет о статусе IP-ядер как показано на рисунке 5. Ядра, требующие перегенерации будут отмечены красным замком, также в графе *«IP Status»* будет написано *«IP revision change»*. По умолчанию ядра, требующие перегенерации будут выбраны галочкой, для автоматического обновления надо нажать кнопку *Upgrade selected»*, отмеченную на рисунке 5 красной рамкой.

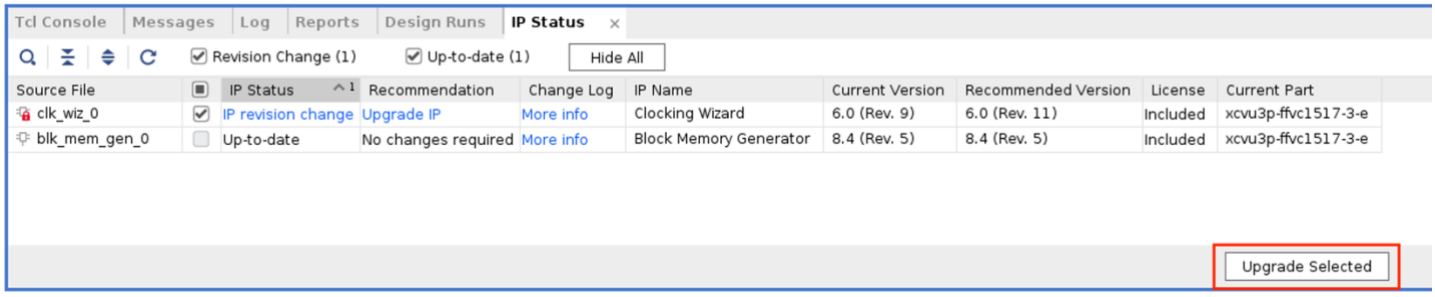


Рисунок 5 — Отчет о статусе IP-ядер

После завершения обновления нужно обновить отчет о статусе IP-ядер. Для этого нажмите кнопку *«Rerun»* как показано на рисунке 6.

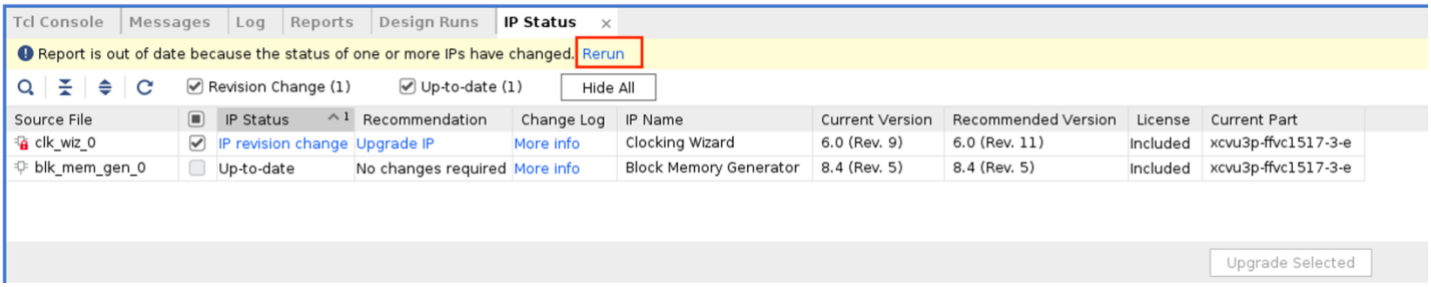


Рисунок 6 — Устаревший отчет о статусе IP-ядер

После обновления отчета вы увидите в графе *«IP Status»* значение *«Up-to-date»* как показано на рисунке 7.

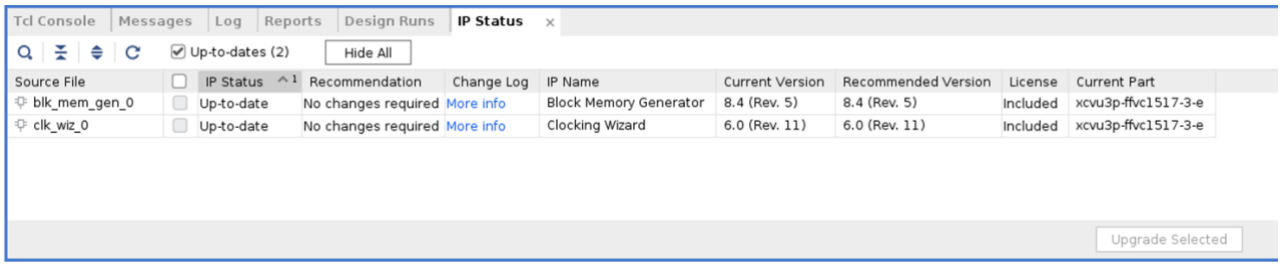


Рисунок 7 — Обновленный отчет о статусе IP-ядер

Если проект был создан в более новой версии, а вы хотите открыть его в старой, среда Vivado предложит вам открыть проект в режиме чтения и пересохранить его для редактирования как показано на рисунке 8.

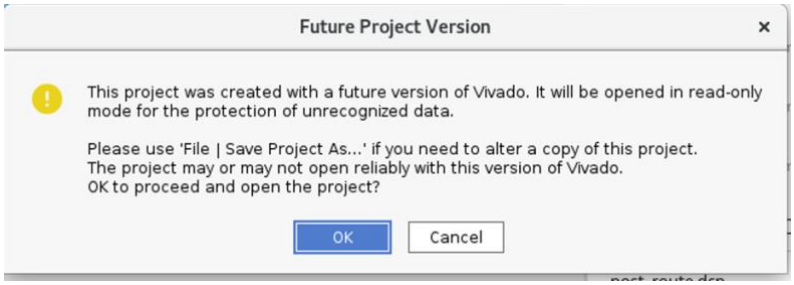


Рисунок 8 — Открытие проекта в старой версии Vivado

Некоторые IP-ядра будет возможно перегенерировать в более старой версии среды если они там присутствовали или версия ядра не изменилась, в обратном случае придется создавать IP-ядра заново, либо использовать альтернативные. Предпочтительно не переходить между версиями в рамках одного выполняемого проекта.

Если вы решили добавить в свой проект IP-ядра, сгенерированные в другой версии Vivado, они также потребуют перегенерации, но среда не предложит сделать это автоматически и потребует от вас проделать это самостоятельно. В дереве проекта такие ядра будут отмечены красным замком как показано на рисунке 9.

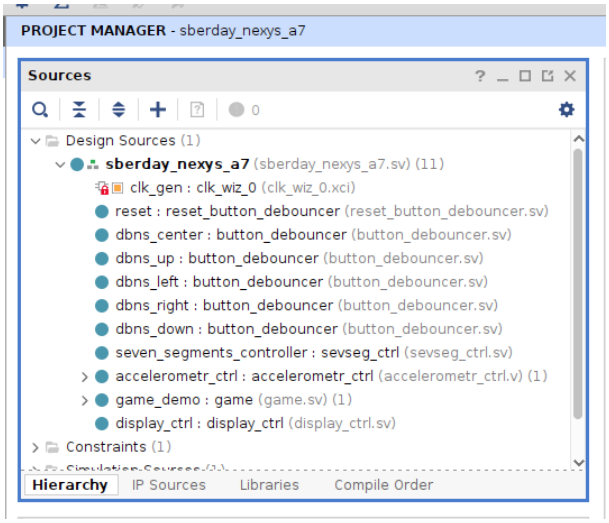


Рисунок 9 — Отображение IP- ядра не актуальной версии

Для обновления ядра нужно нажать на него правой кнопкой мыши и выбрать пункт «Upgrade IP ...» как показано на рисунке 10.

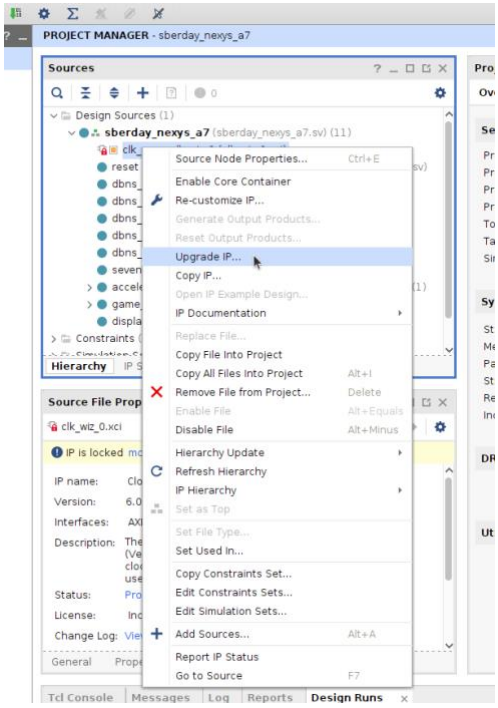


Рисунок 10 — Обновление IP-ядра

После выполнения обновления Vivado предложит сгенерировать генерируемые файлы для выбранной IP, после чего можно использовать ядро в проекте.

2. Описание графического интерфейса среды Vivado

Для базового освоения навыков работы со средой разработки Vivado лучше всего подходит маршрут разработки в графическом интерфейсе, так как в таком режиме многие настройки скрыты от пользователя и в случае не больших проектов, работа сильно упрощена. На рисунке 11 приведено изображение графического интерфейса среды Vivado.

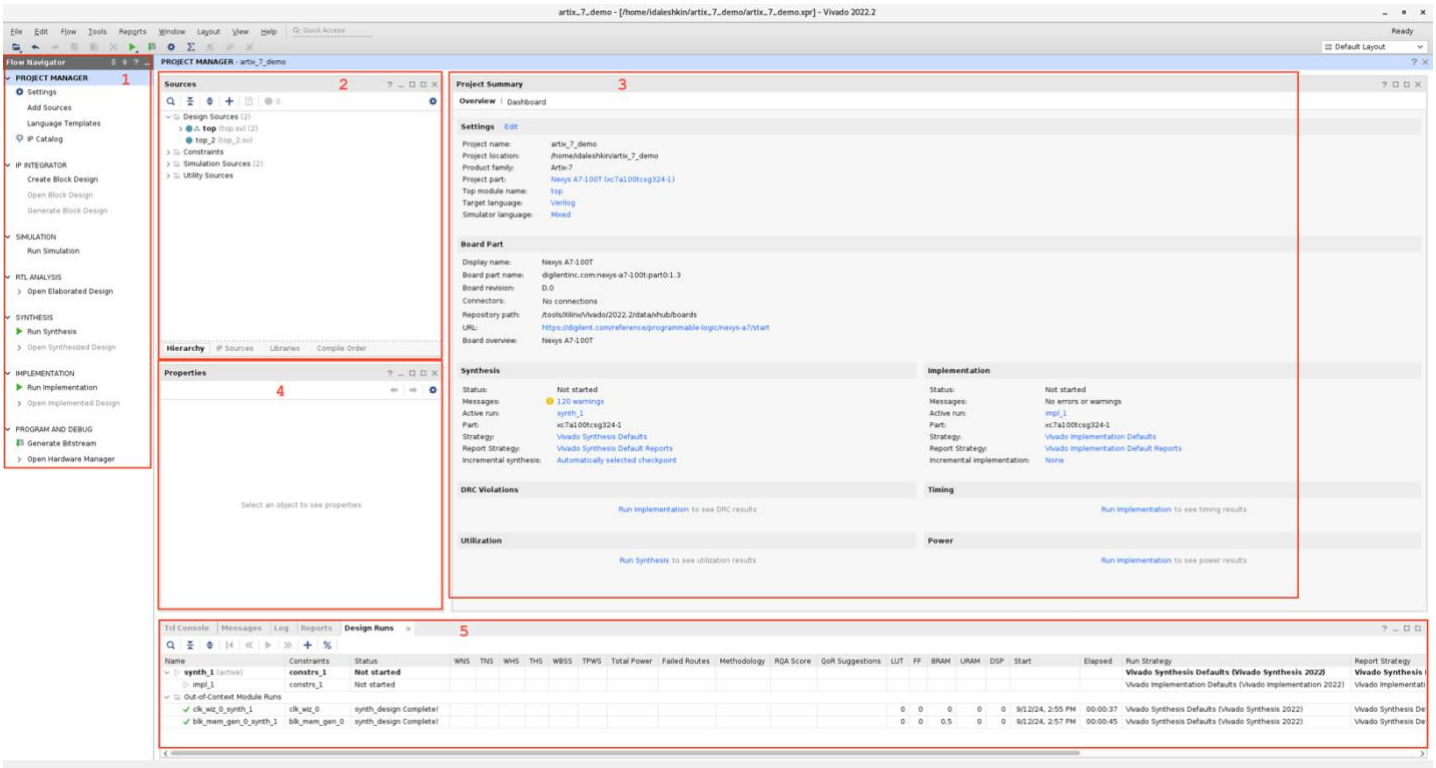


Рисунок 11 — Графический пользовательский интерфейс AMD(Xilinx) Vivado

Можно подразделить элементы интерфейса на несколько пунктов:

1) Окно *«Flow Navigator»* — это инструмент, который предоставляет контроль над основными функциями среды, такими, как настройка проекта, синтез, имплементация и генерация битстрима. Доступные команды и параметры в *«Flow Navigator»* зависят от статуса сборки дизайна, недоступные шаги будут обозначены серым цветом до тех пор, пока не будут выполнены необходимые этапы сборки. Далее более подробно будут рассмотрены пункты *«RTL Analysis»*, *«Synthesis»*, *«Implementation»* и *«Program and Debug»*.

Другие пункты в рамках вашего задания не необходимы, а настройки проекта формируются автоматически. Стоит отметить, что вам может быть полезен раздел *«Language Templates»*, там приведены подсказки по синтаксису, примеры модулей, а также поддерживаемые средой разработки Vivado конструкции HDL — языков.

2) Окно *«Sources»* — показывает иерархию файлов проекта. Среда разработки автоматически распознает иерархию проекта анализируя добавленные файлы. При изменении содержимого файлов среда обновляет статус файлов проекта в окне *«Sources»*. В пункте *«IP Sources»* отображаются добавленные в проект IP — ядра. В пункте

«Libraries» файлы отсортированы по типам и библиотекам. В пункте *«Compile Order»* отображается порядок, в котором файлы будут скомпилированы средой;

3) Окно *«Project Summary»* — краткая сводка о проекте, отражает основные моменты о настройках проекта, состоянии синтеза и имплементации;

4) Окно *«Properties»* — отображает информацию о выбранном в дереве проекта файле, как приведено на рисунке 12;

5) Окно с инструментами анализа, логами, визуализатором репортов.

- *«TCL Console»* позволяет управлять средой с помощью команд вручную, а также посмотреть, как среда преобразует действия пользователя в GUI в TCL команды;
- Графа *«Messages»* позволяет посмотреть на полученные в результате сборки ошибки, предупреждения, а также обычные информационные сообщения в более комфортном виде, с делением на этапы, ссылками, кодами ошибок и предупреждений;
- В графе *«Logs»* приведены логи отдельно для каждого этапа сборки;
- В пункте *«Reports»* можно сгенерировать и посмотреть различные отчеты о результатах этапов сборки;
- Графа *«Design Runs»* позволяет отслеживать, на каком этапе находится сборка проекта, какие стратегии синтеза и имплементации применяются, отслеживать время, затраченное на каждый этап, временные нарушения по setup и hold, а также потребляемые ресурсы.

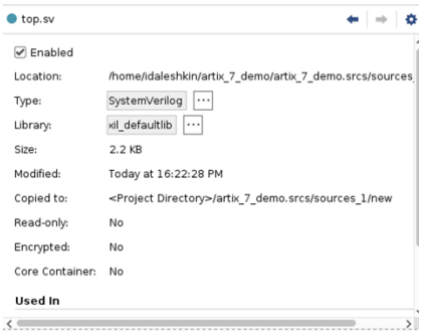


Рисунок 12 — Поле *«Properties»* в GUI Vivado

3. Работа с файлами проекта

Рассмотрим, как можно добавить файл в проект. Для этого в поле *«Sources»* нужно нажать на значок *«+»* как показано на рисунке 13.

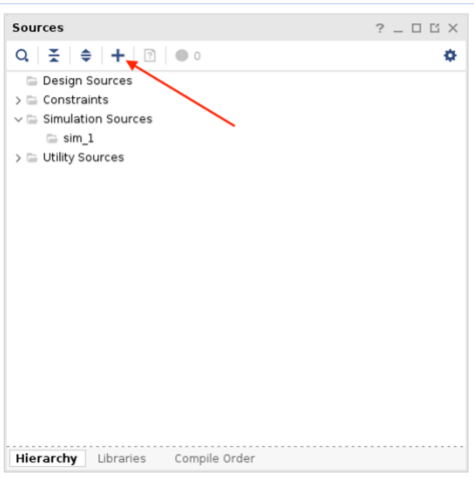


Рисунок 13 — Кнопка для добавления файла в проект

Далее выбрать пункт *«Add or create design sources»* как показано на рисунке 14 и нажать *«Next»*.

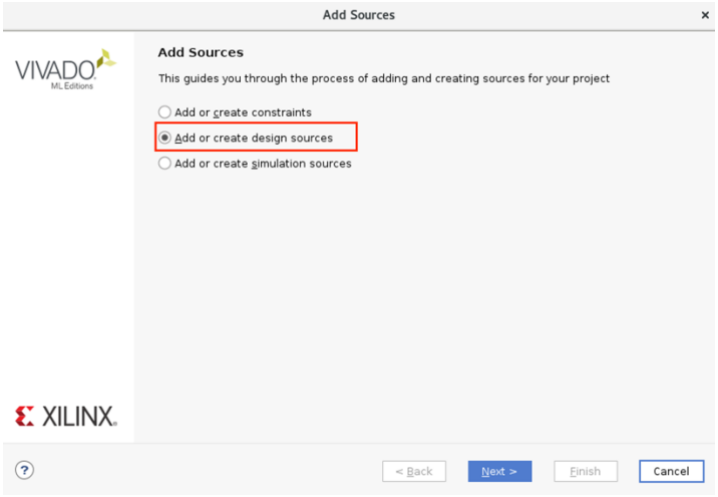


Рисунок 14 — Окно выбора категории файла для добавления в проект

Далее нажать на кнопку *«Add files»* если вы хотите добавить уже имеющийся файл, либо нажать *«Create file»* если хотите создать новый файл, как показано на рисунке 15.

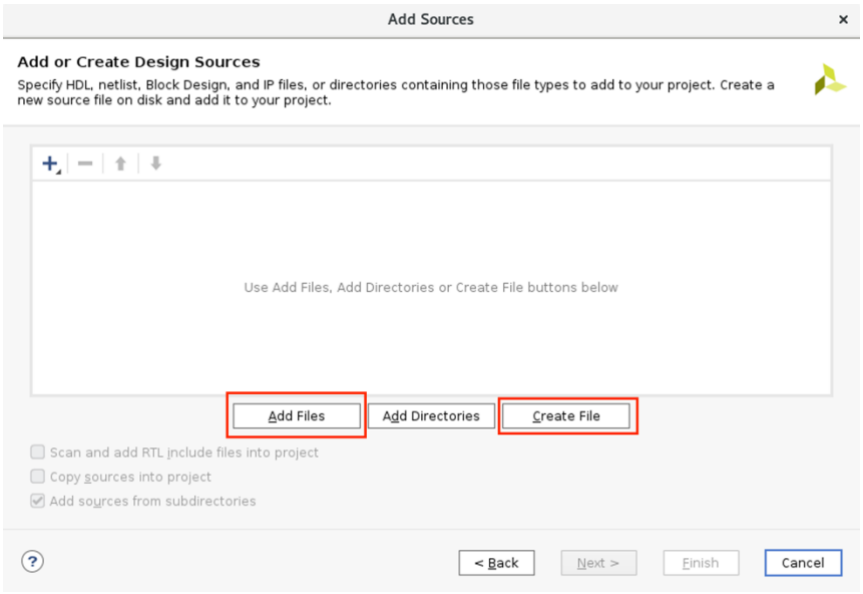


Рисунок 15 — Окно добавленных файлов

В случае если вы нажали *«Add files»*, появится поле для выбора файла как показано на рисунке 16.



Рисунок 16 — Окно выбора файов

Если вы выбрали *«Create File»*, появится поле с возможностью выбора типа файла, ввода имени файла и выбора расположения файла как показано на рисунке 17. Если в последней графе вы оставите *<Local to Project>*, то файл будет создан в папке */<имя_проекта>.srcs/sources_1/new/* .

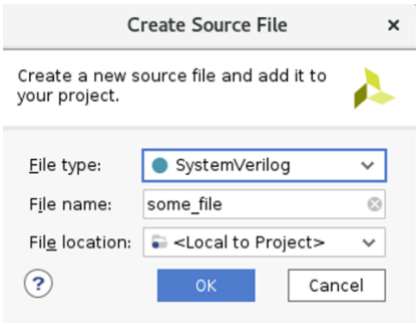


Рисунок 17 — Окно создания нового файла проекта

После нажатия *«OK»*, появится окно со списком добавленных файлов как показано на рисунке 18.

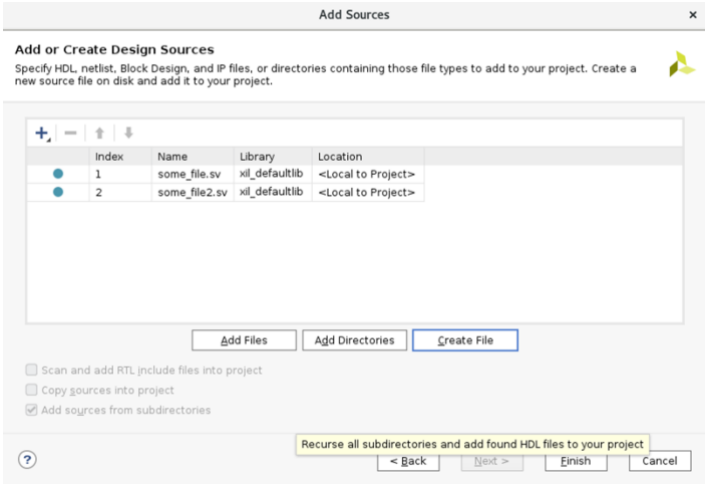


Рисунок 18 — Окно с добавленными файлами

После нажатия *«Finish»*, среда позволяет выбрать для созданных новых файлов название модуля внутри файла, задать направления портов и их названия для генерации шаблона, интерфейс окна показан на рисунке 19.

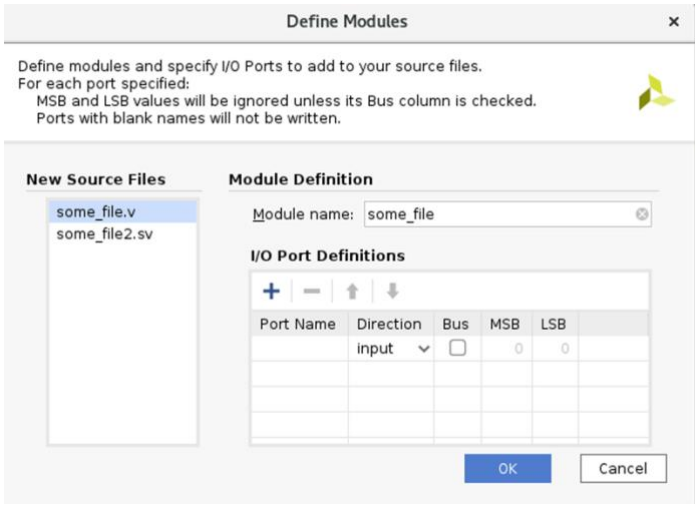


Рисунок 19 — Окно задания параметров шаблона для новых файлов

После нажатия *«OK»,* файлы будут добавлены в проект. В окне *«Sources»* в дереве проекта после инстанцирования новых добавленных либо созданных модулей в модуль *«top»* в файле *«top.sv»*, Vivado автоматически выстраивает дерево проекта (Рисунок 20).



Рисунок 20 — Дерево проекта в окне *«Sources»*

Для удаления файла из проекта нужно нажать на желаемый файл правой кнопкой мыши и выбрать *«Remove File from Project»* как изображено на рисунке 21а. Также есть возможность отключить файл не удаляя из проекта, нажав *«Disable file»* (рисунок 21б), позже при необходимости можно будет вновь его включить.

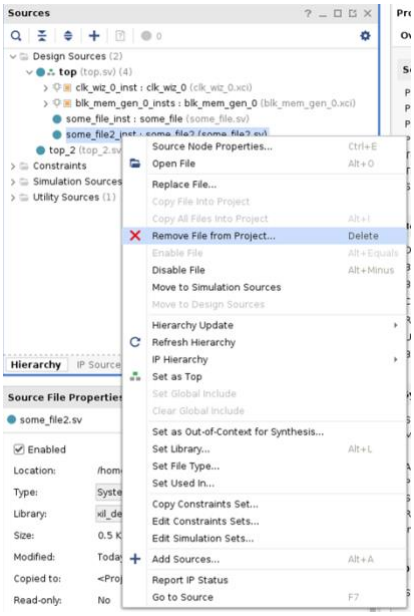


Рисунок 21а — Удаление файла из проекта Vivado

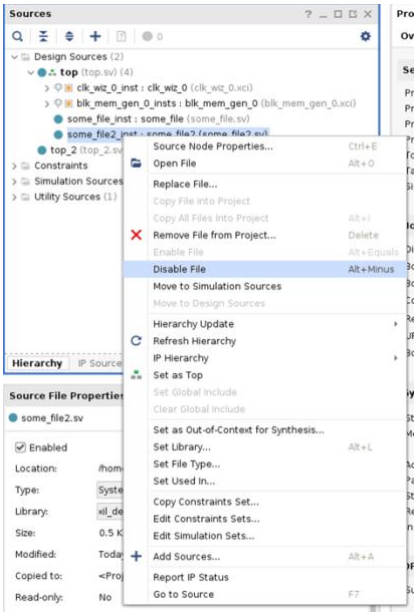


Рисунок 21б — Отключение файла в проекте

Иногда среда некорректно автоматически определяет файл верхнего уровня, но можно назначить его, нажав правой кнопкой мыши на нужный файл и затем нажав на пункт *«Set as Top»* как показано на рисунке 22.

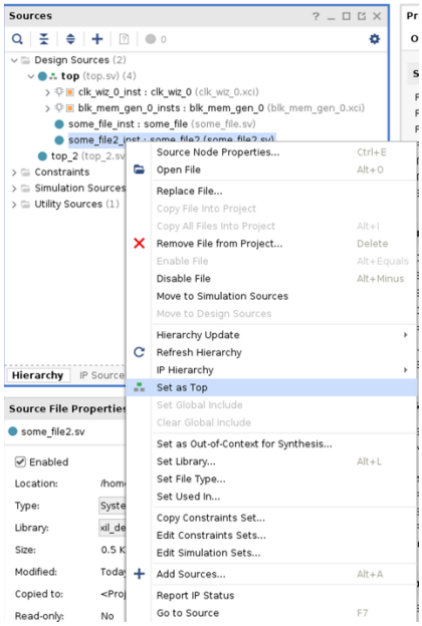


Рисунок 22 — Назначение файла в качестве верхнего уровня проекта

4. Работа с IP — ядрами, доступными в среде Vivado

В среде AMD(Xilinx) Vivado вендор предоставляет пользователю возможность использовать некоторые готовые модули, оптимизированные под используемую архитектуру ПЛИС в каждом конкретном случае. Такие модули в основном реализуют простые, часто используемые функции, либо стандартизованные сложные подсистемы. Они называются IP — ядрами.

Для работы с IP — ядрами в Vivado существует раздел *«IP Catalog»*. В нем есть ядра как предоставляемые средой, так и платные, предоставляемые различными вендорами. Кнопка активации *«IP Catalog»* приведена на рисунке 23.

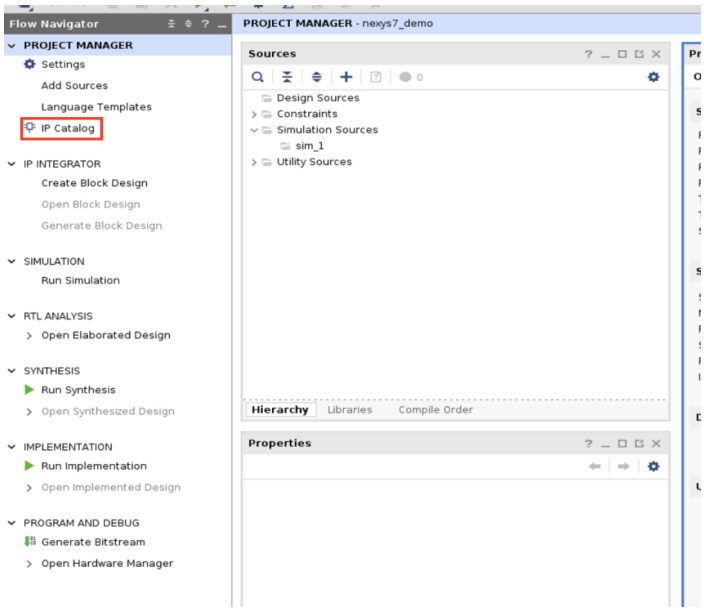


Рисунок 23 — Кнопка активации IP catalog

Для выполнения вашего проекта, а также упрощения проектирования предлагается задействовать следующие IP:

- Clocking Wizard — для формирования тактовых сигналов в вашем проекте;
- Block Memory Generator — для формирования памятей различной архитектуры на основе примитива блочной памяти.

Описание конфигурации IP — ядер приведено в следующих разделах.

4.1. Clocking Wizard

IP — ядро *«Clocking Wizard»* упрощает работу с генерацией тактовых сигналов. Интерфейс генератора IP — ядер позволяет переопределять параметры ядра в соответствии с требованиями пользователя.

Ядро *«Clocking Wizard»* состоит из выбранного примитива для работы с тактовым сигналом и вспомогательной логики. Структурная схема ядра *«Clocking Wizard»* приведена на рисунке 24. Подробное описание можно найти в источнике [1].

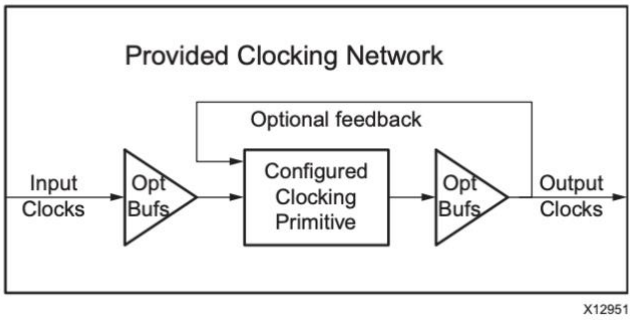


Рисунок 24 — Структурная схема ядра *«Clocking Wizard»*

Для выбора нужного IP — ядра в меню Vivado *«Flow Navigator»* нужно выбрать пункт *«IP Catalog»*, как изображено на рисунке 25.

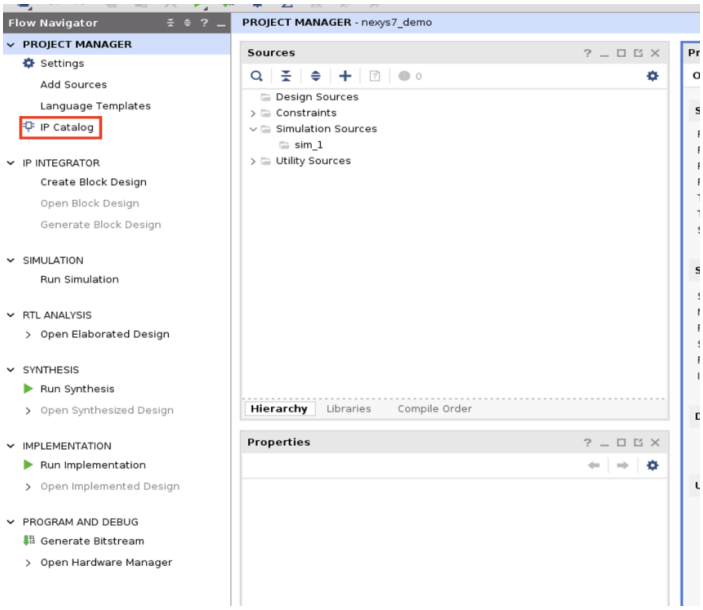


Рисунок 25 — Интерфейс среды AMD(Xilinx) Vivado

Далее в строке поиска ввести название IP: *«Clocking Wizard»*. Список в *«IP Catalog»* сократится до содержащих введенное название, далее нужно выбрать *«Clocking Wizard»* IP как показано на рисунке 26.

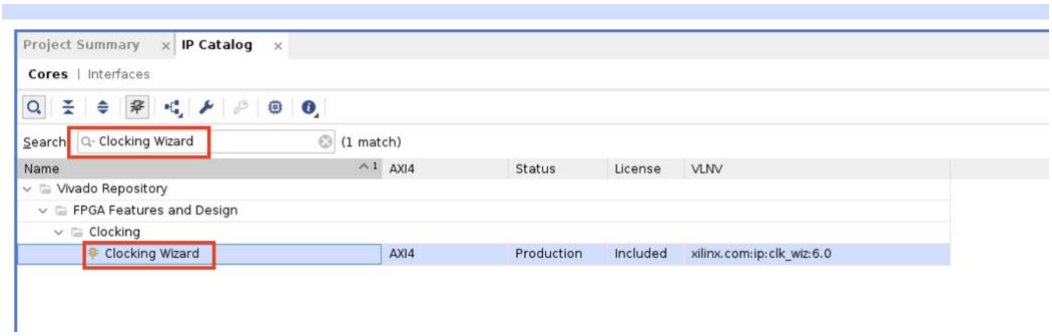


Рисунок 26 — Интерфейс *«IP Catalog»* с поисковым запросом *«Clocking Wizard»*

Прежде чем конфигурировать ядро, необходимо узнать из документации на вашу платформу параметры внешнего тактового сигнала, в случае с отладочной платой Nexys A7, это можно посмотреть в документации на плату[2]. На рисунке 27 приведена выдержка с описанием тактового сигнала.

5 Oscillators/Clocks

The Nexys A7 board includes a single 100 MHz crystal oscillator connected to pin E3 (E3 is a MRCC input on bank 35). The input clock can drive MMCMs or PLLs to generate clocks of various frequencies and with known phase relationships that may be needed throughout a design. Some rules restrict which MMCMs and PLLs may be driven by the 100 MHz input clock. For a full description of these rules and of the capabilities of the Artix-7 clocking resources, refer to the “7 Series FPGAs Clocking Resources User Guide” available from Xilinx.

Xilinx offers the Clocking Wizard IP core to help users generate the different clocks required for a specific design. This wizard will properly instantiate the needed MMCMs and PLLs based on the desired frequencies and phase relationships specified by the user. The wizard will then output an easy-to-use wrapper component around these clocking resources that can be inserted into the user’s design. The clocking wizard can be accessed from within the Project Navigator or Core Generator tools.

Рисунок 27 — Скриншот раздела о тактовом сигнале из документации на отладочную плату Nexys A7

Также необходимо посмотреть на схеме электрической принципиальной[3] модель тактового генератора, скриншот со схемы приведен на рисунке 28.

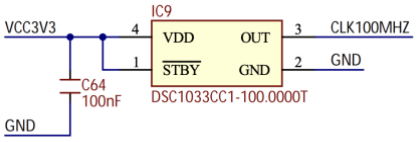


Рисунок 28 — Тактовый генератор на схеме электрической принципиальной отладочной платы Nexys A7

И посмотреть в его документации значение джиттера в зависимости от частоты, приведено на рисунке 29.

Input logic low	V_{IL}				0.25 V_{DD}	
Jitter, Cycle to Cycle	J_{CC}	F = 100MHz[3]		95		ps

Notes:
1. Absolute maximum ratings are those values beyond which the safety of the device cannot be guaranteed. The device should not be operated beyond these limits.
2. t_{SU} is time to stable output frequency after V_{DD} is applied. t_{SU} and t_{EN} (after EN is asserted) are identical values.
3. See typical cycle to cycle jitter graph for frequency dependence.

Рисунок 29 — Значение джиттера при частоте 100 МГц для используемого тактового генератора

На рисунке 30 представлен интерфейс настройки ядра. Пункты настройки обозначены на рисунке красными цифрами.

Пункт 1: включение мониторинга состояния входных тактовых сигналов через AXI Lite интерфейс. Данная функция позволяет контролировать изменение частоты тактового сигнала относительно заданной, остановку тактового сигнала и сбои;

Пункт 2: выбор примитива, используемый в ядре: MMCM (mixed mode clock manager), PLL (phase locked loop). В нашем случае можно оставить MMCM по умолчанию;

Пункт 3: позволяет использовать различные дополнительные возможности, о которых можно подробно узнать в документации от вендора [1];

Пункт 4: необходимо ввести параметры входных тактовых сигналов:

1. При необходимости включить вторичный тактовый сигнал, в вашем случае не используется;
2. Ввести название порта для входного тактового сигнала либо оставить предложенное средой;
3. Ввести частоту входного тактового сигнала, в случае с платой Nexys A7 — 100 МГц;
4. Выбрать единицы измерения джиттера: unit intervals или пикосекунды, в документации указаны пикосекунды, поэтому указываем в пикосекундах;
5. Ввести значение джиттера из документации — 95 ps;
6. Также нужно указать тип источника тактового сигнала, в нашем случае в документации описано, что тактовый сигнал подается на пин E3, значит тактовый сигнал не дифференциальный и пин E3 является MRCC пином (Multi region clock capable), поэтому выбрана настройка *«Single ended clock capable pin»*.

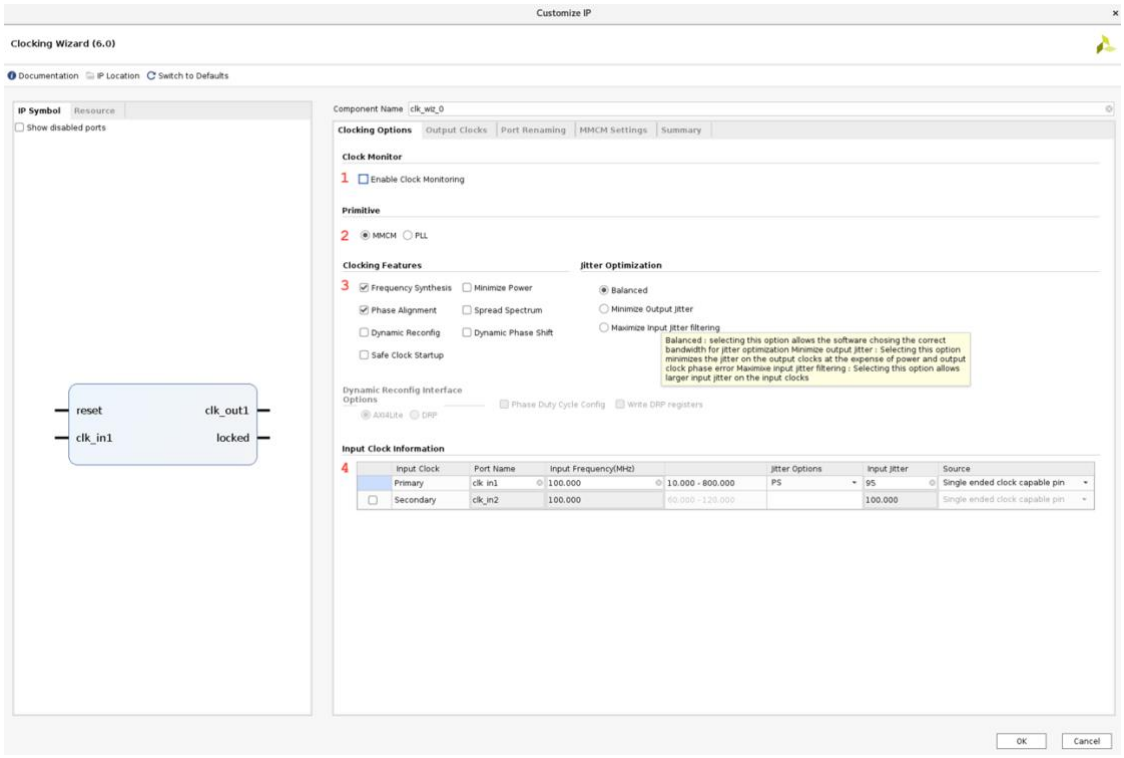


Рисунок 30 — Интерфейс генератора IP — ядра Clocking Wizard

На рисунке 31 приведена страница с настройками выходных тактовых сигналов. Цифрой *«1»* обозначено поле, где можно выбрать количество тактовых сигналов, названия портов и их частоту, однако стоит учитывать, что среда производит пересчет значения указанных вами частот в коэффициенты для входной частоты с учетом допустимой частоты работы генератора, управляемого напряжением (ГУН (англ. VCO)), поэтому не все значения могут быть получены, так как значения коэффициентов деления и умножения частоты не произвольны и ограничены, также частота ГУНа не произвольна и имеет ограничения. В случае если выбранную вами частоту невозможно получить из исходной, среда подсветит значение красным и предложит ближайшую доступную.

Цифрой *«2»* обозначено поле, где можно выбрать опциональные порты, полярность сброса и режим сдвига фазы. Рекомендуется оставить порты *«reset»* и *«locked»*, использовать последний при формировании сброса вашей логики в соответствующем тактовом домене.

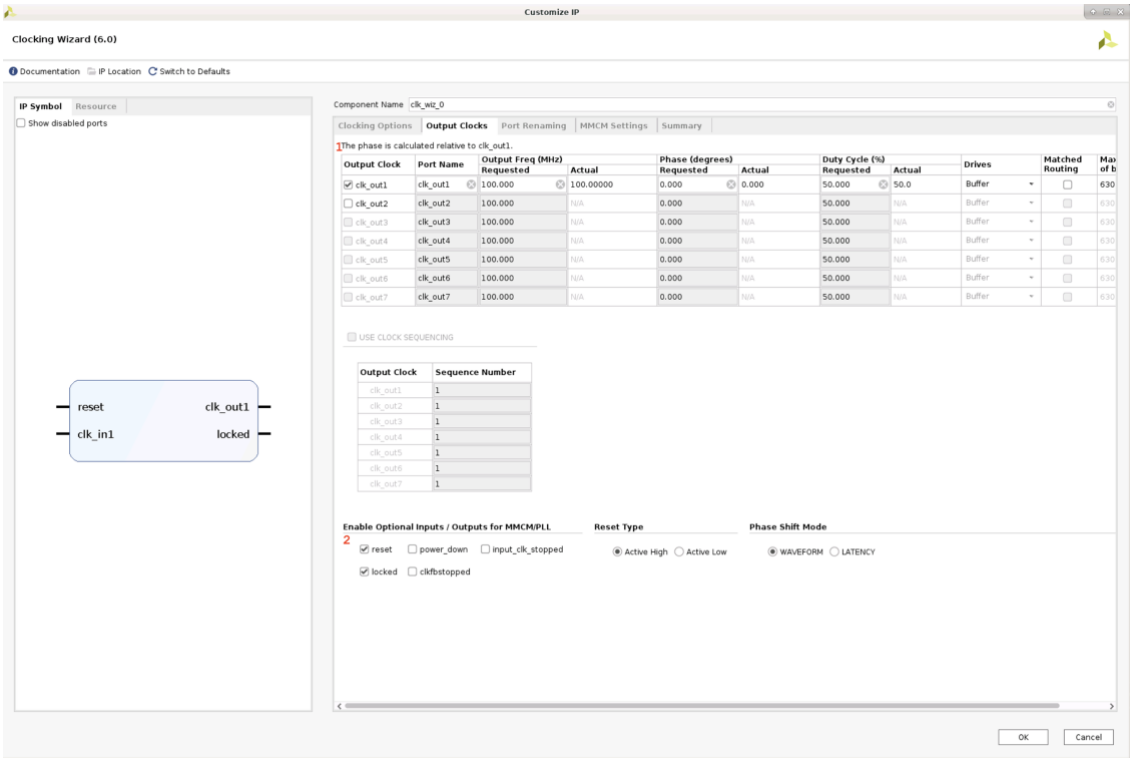


Рисунок 31 — Настройка выходных тактовых сигналов

На рисунке 32 представлена страница, позволяющая посмотреть расчетное значение ГУНа (VCO), а также переназвать опциональные порты, выбранные на предыдущей странице.

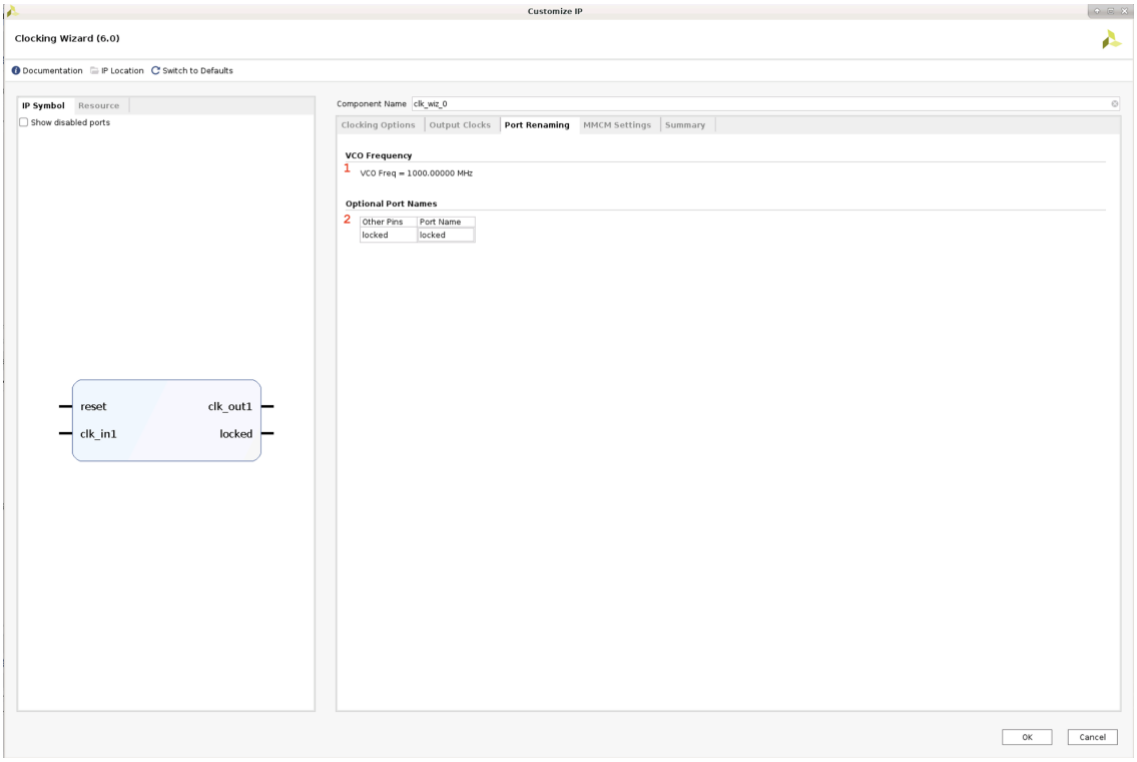


Рисунок 32 — Настройка названий опциональных портов

На рисунке 33 приведена страница с полученными из введенных настроек конфигурациями выбранного примитива.

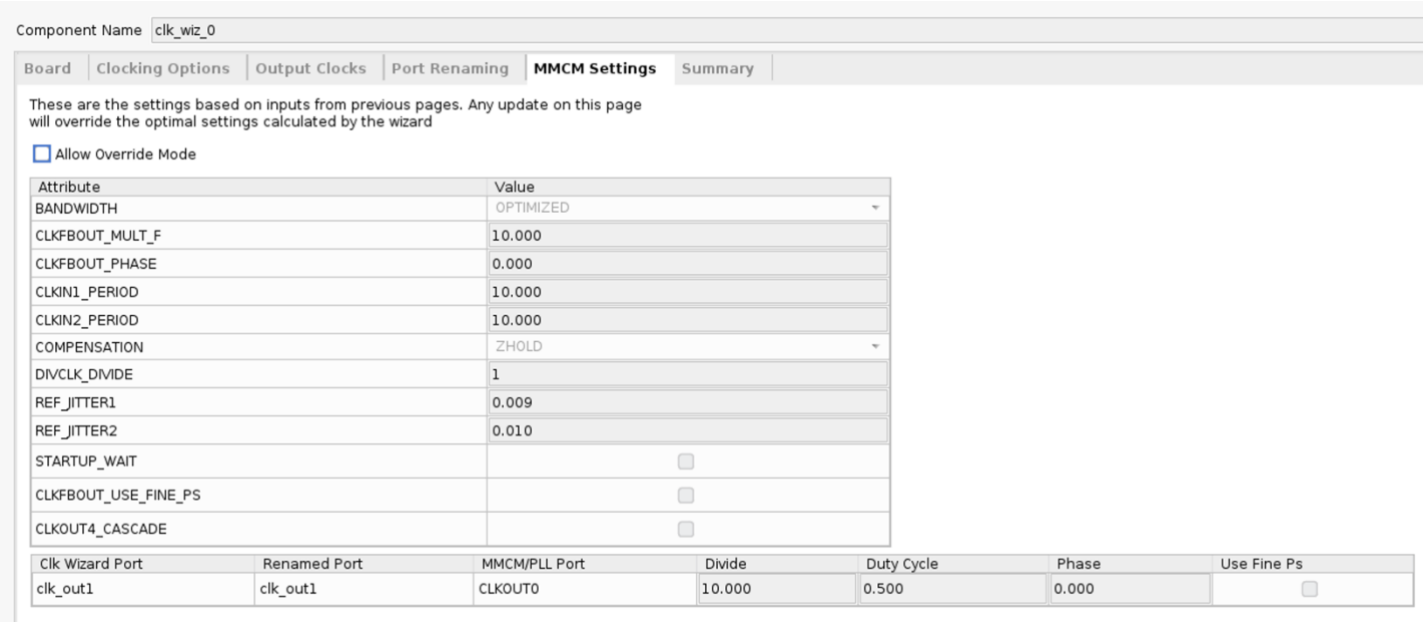


Рисунок 33 — Настройки используемого для формирования тактового сигнала примитива

На рисунке 34 приведена страница с краткой сводкой по результатам настойки.

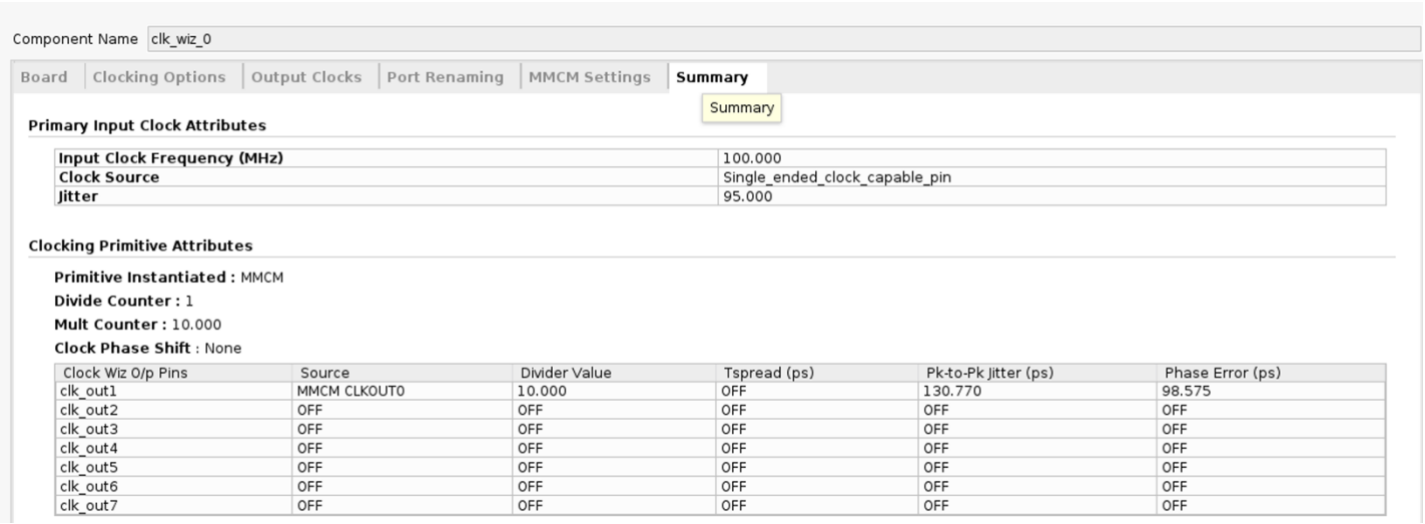


Рисунок 34 — Отчет по настройкам ядра исходя из предыдущих пунктов

После настройки и нажатия *«OK»* появится окно как на рисунке 35. Пункт *«Synthesis Options»* позволяет настроить тип синтеза IP — ядра, Out-Of-Context синтез подразумевает, что ядро синтезируется отдельно, а при общем синтезе проекта используется ранее заготовленный результат синтеза ядра, это позволяет ускорить синтез проекта за счет того, что не надо каждый раз при новом синтезе проекта пересинтезировать ядро. Global же генерирует файл описания ядра, используемый каждый раз при новом синтезе проекта для пересинтезирования

IP — ядра. Дальнейшие настройки позволяют настроить, на локальной или удаленной машине проводить синтез, либо сгенерировать скрипт для автоматизированного синтеза.

Для продолжения требуется нажать *«Generate»*.

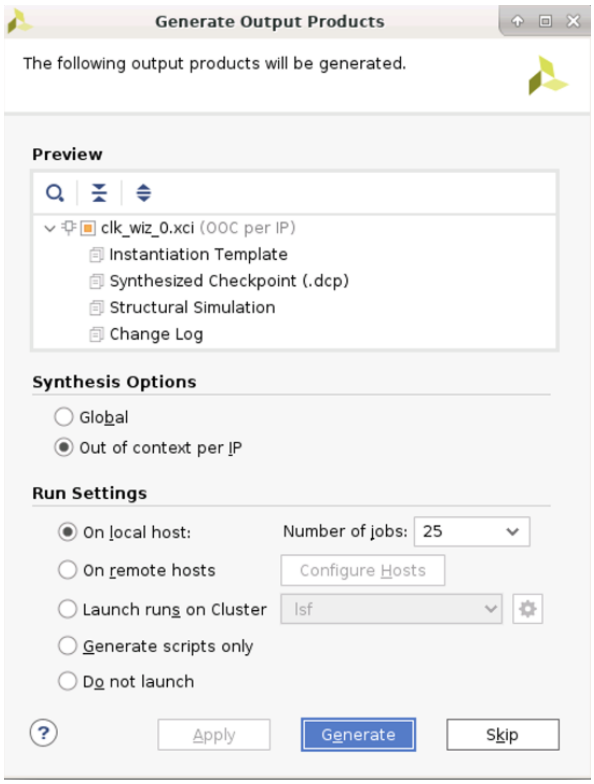


Рисунок 35 — Окно генерации IP — ядра

Сгенерированное IP — ядро должно появиться в окне иерархии проекта во вкладке *«IP Sources»* как показано на рисунке 36.



Рисунок 36 — Список IP — ядер в проекте.

Далее можно развернуть IP — ядро как показано на рисунке 37 и посмотреть, что сгенерировала среда, там будут файлы с шаблонами для инстанцирования IP в ваш HDL — код, файлы для симулятора и результаты *«Out-Of-Context»* синтеза в виде *.dcp файла.

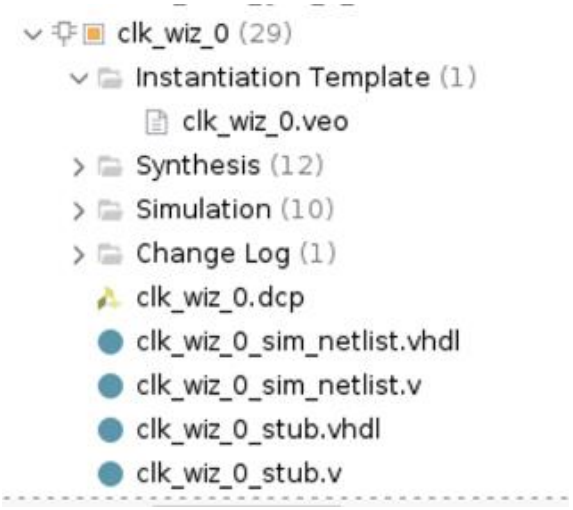


Рисунок 37 — Сгенерированные файлы ядра.

В директории с сгенерированными файлами вы можете увидеть файлы с названиями, не похожими на название IP — ядра, например, в данном случае будут сгенерированы файлы, относящиеся к примитиву MMCM, который использует IP ядро как показано на рисунке 38, это не вызвано никакими ошибками среды, так и должно быть.

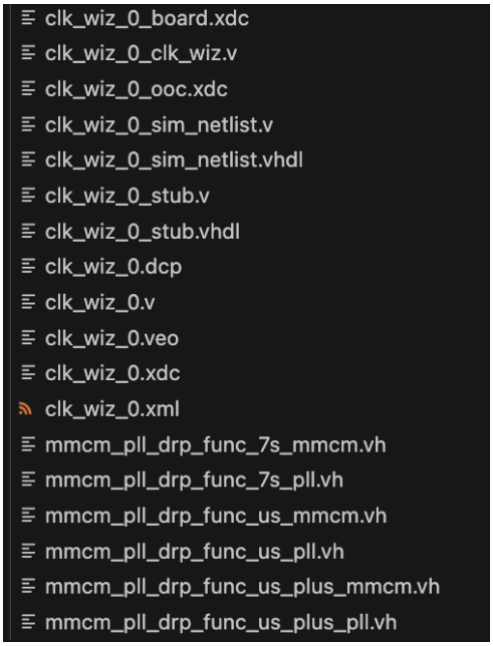


Рисунок 38 — Сгенерированные файлы в директории IP — ядра

4.2. Block Memory Generator

IP — ядро Block Memory Generator позволяет генерировать оптимизированную по ресурсам и производительности память на основе примитива встроенной в ПЛИС блочной памяти (BRAM). Ядро

поддерживает как нативный, так и AXI4 интерфейс. Подробные сведения об этом IP — ядре можно посмотреть в источнике

Для выбора нужного IP — ядра в меню Vivado слева нужно выбрать пункт *«IP Catalog»* как показано на рисунке 39.

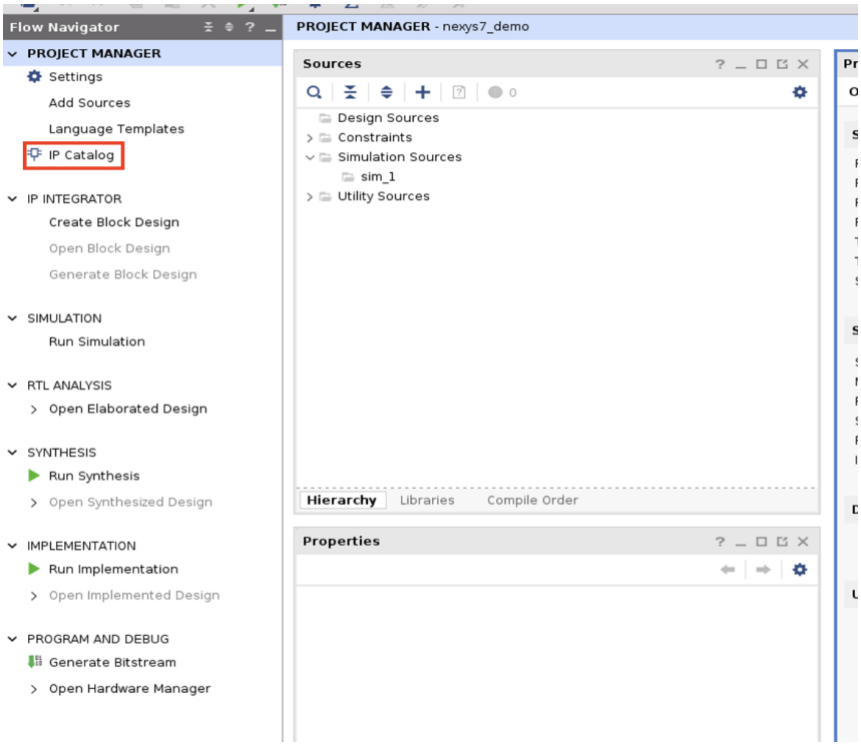


Рисунок 39 — Интерфейс среды Xilinx Vivado

Далее в строке поиска ввести название IP: Block Memory Generator. Список в IP Catalog сократится до содержащих введенное название, далее нужно выбрать Block Memory Generator как на рисунке 40.

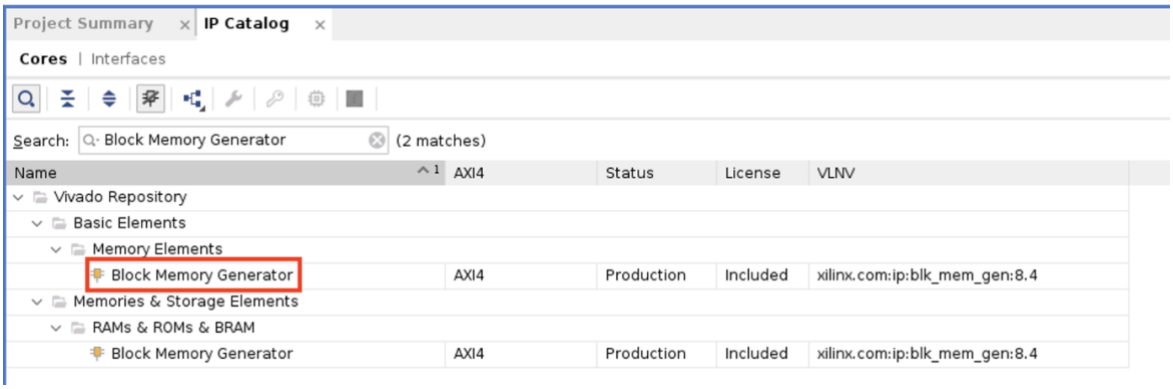


Рисунок 40 — Интерфейс IP Catalog

То, что данное IP — ядро появилось в нескольких категориях на рисунке 40 не означает, что это разные IP — ядра, оно попадает под несколько категорий поиска, можно выбрать любое.

Далее перейдем к настройкам ядра, страница базовых настроек изображена на рисунке 41, красными цифрами обозначены пункты настроек.

Пункт 1: выбор интерфейса подключения: нативный либо AXI4. Также можно выбрать тип памяти, который требуется сгенерировать: Single Port RAM, Simple Dual Port RAM, True Dual Port RAM, Single Port ROM, Dual Port ROM. Также можно включить приведение шины адреса к 32 разрядному виду и выбрать для двухпортовой памяти, общее ли тактирование использовать для обоих портов либо каждому порту свое. Настройки зависят от того, как вы хотите использовать память.

Пункт 2: настройка использования кода, исправляющего ошибки (ECC)

Пункт 3: включение побайтового разрешения записи в память.

Пункт 4: выбор алгоритма объединения примитивов блочной памяти, от этого будет зависеть, какой из примитивов будет использоваться для генерации.

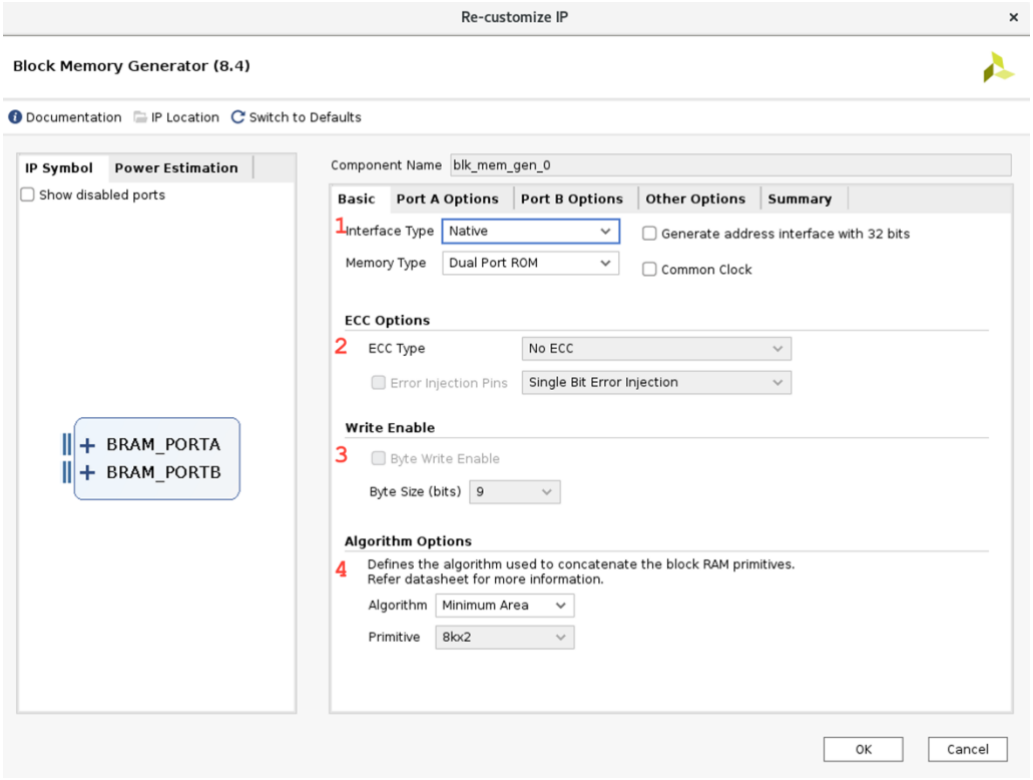


Рисунок 41 — Базовые настройки IP — ядра Block Memory Generator

На рисунке 42 приведены настройки порта A генерируемой памяти, пункты обозначены красными цифрами.

Пункт 1: Выбор ширины и глубины памяти, диапазон значений указан рядом с полем ввода. Также можно выбрать режим работы памяти: Read-First, Write-First или No Change. И можно настроить использование сигнала ENA, включающего порт;

Пункт 2: включение регистров на выходной шине данных на выходе примитива и на выходе ядра;

Пункт 3: не доступен для ПЛИС Xilinx 7 серии.

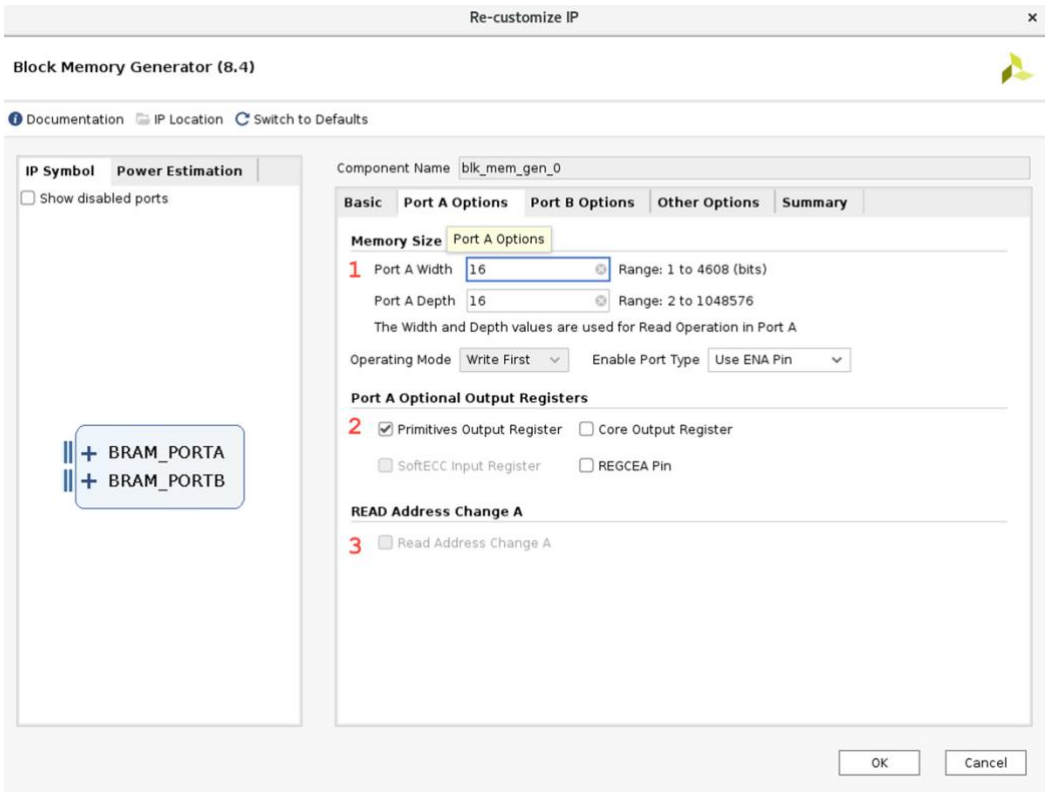


Рисунок 42 — Настройки порта A

Настройки порта B аналогичны настройкам порта A и приведены на рисунке 43, но если выбрана однопортовая память, страницы для порта B не будет.

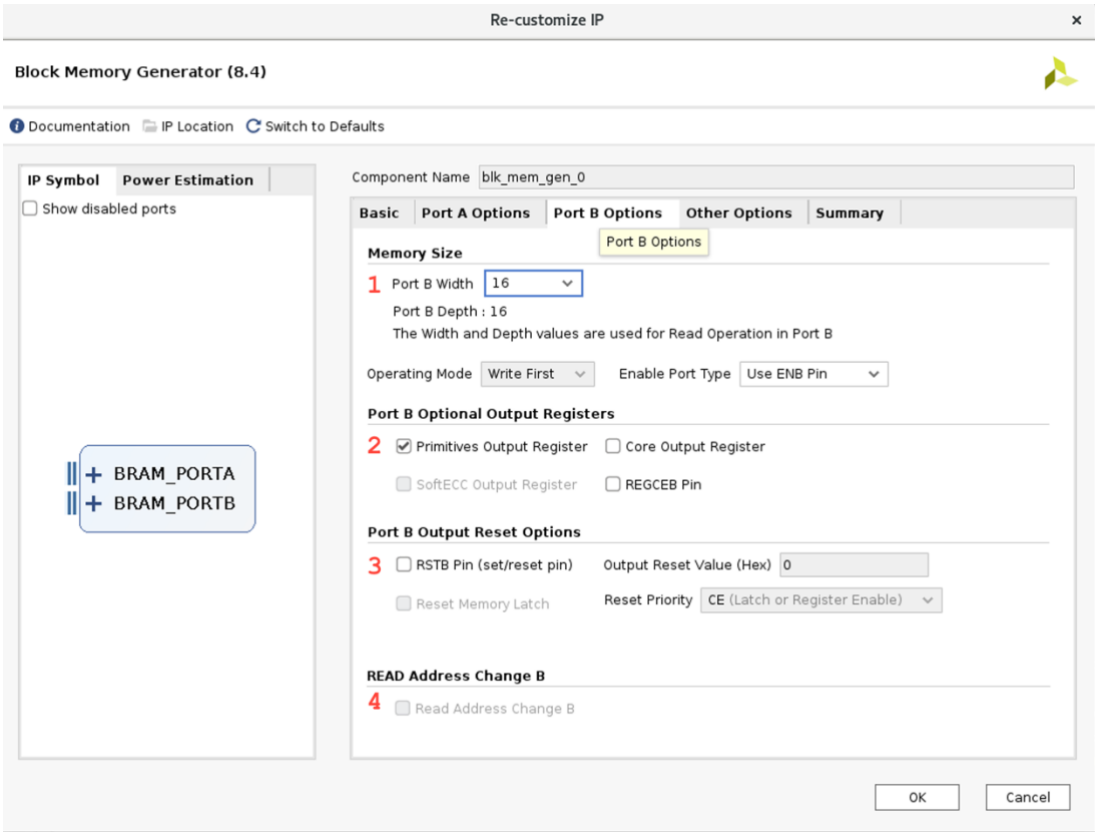


Рисунок 43 — Настройки порта B

На рисунке 44 приведены прочие настройки ядра, описание пунктов приведено далее.

Пункт 1: настройка конвейеризации выходной шины данных для повышения производительности ядра, но данная настройка работает только при включенном регистре на выходной шине данных.

Пункт 2: настройка начальной инициализации памяти, из COE — файла, либо инициализация всей памяти заданным значением. Примеры синтаксиса COE — файла можно посмотреть по ссылке: https://docs.amd.com/r/en-US/ug896-vivado-ip/COE-File-Syntax .

Пункт 3: настройка для симуляционной модели, которая задает генерацию предупреждений в случае коллизий при работе с памятью и значение на выходной шине данных в случае коллизий.

Пункт 4: отключение предупреждений для симуляционной модели.

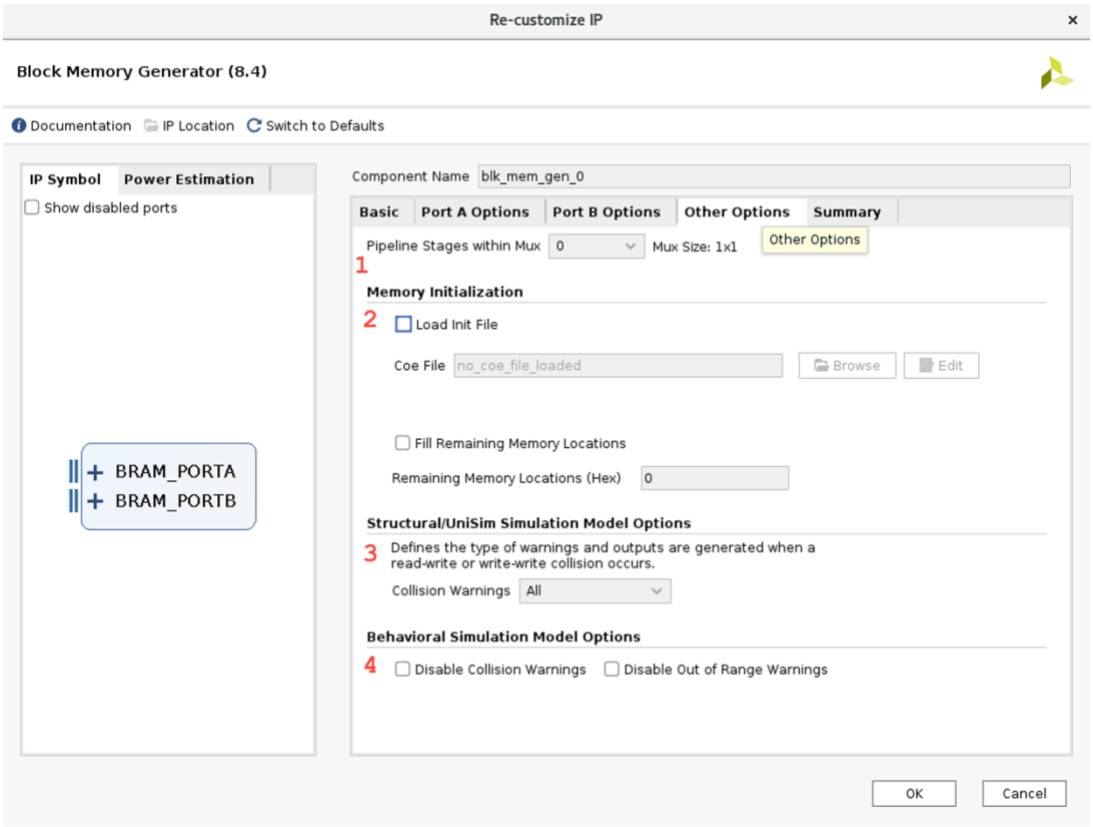


Рисунок 44 — Прочие настройки

На рисунке 45 приведена краткая сводка по настройке IP — ядра Block Memory Generator. В данном случае показаны тип памяти, тип и количество используемых примитивов, задержка на портах и ширина адресных шин каждого из портов.

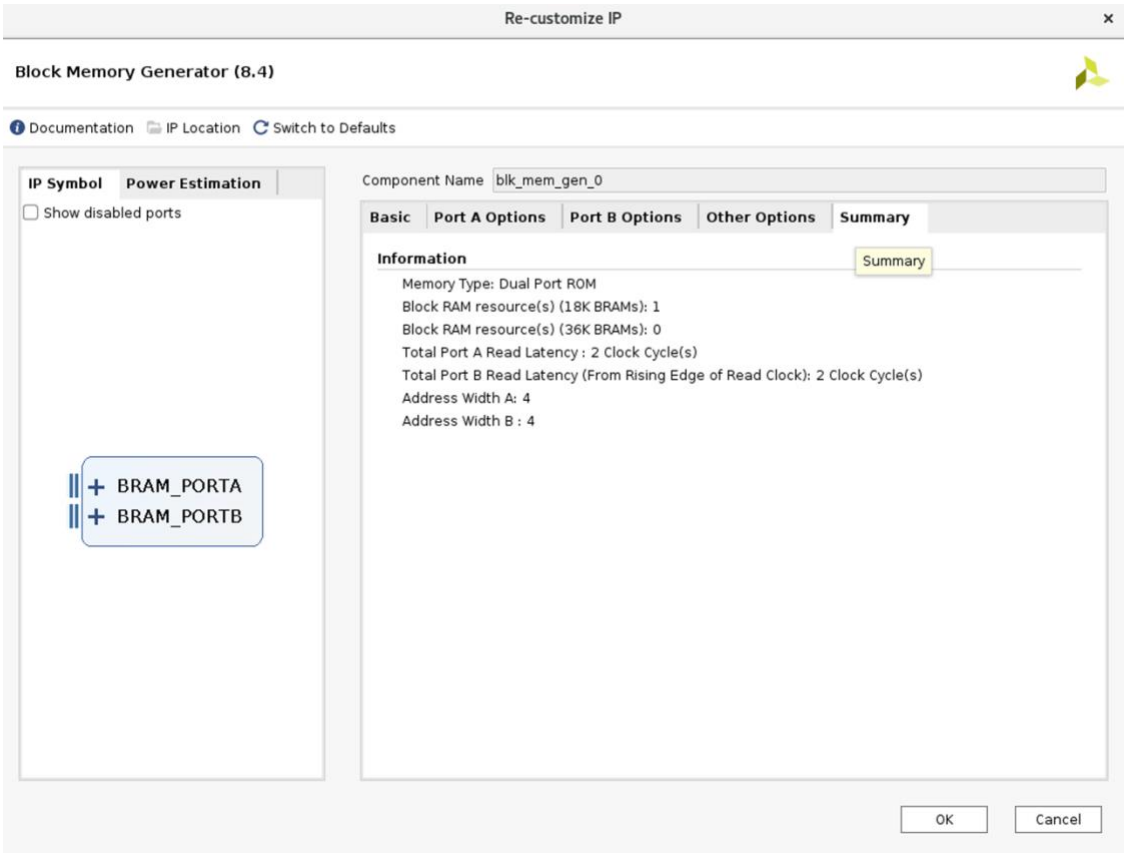


Рисунок 45 — Краткая сводка по заданным настройкам

После настройки и нажатия *«OK»* появится окно как на рисунке 46.

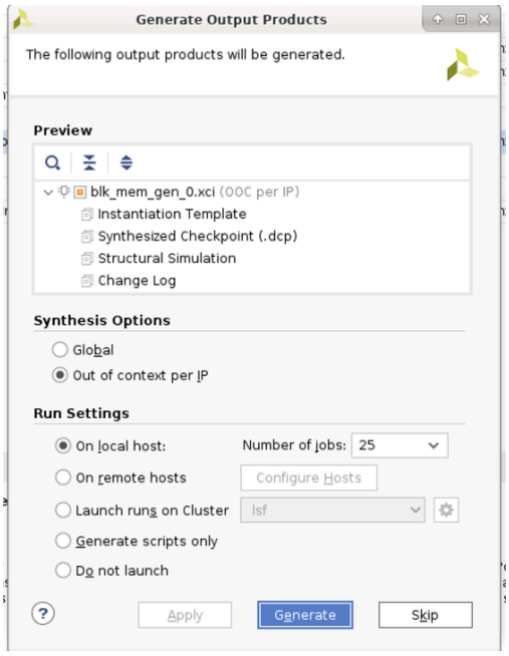


Рисунок 46 — Окно генерации IP — ядра

Пункт *«Synthesis Options»* позволяет настроить тип синтеза IP — ядра, *«Out-Of-Context»* синтез подразумевает, что ядро синтезируется отдельно от всего проекта, а при общем синтезе проекта используется

ранее заготовленный результат синтеза ядра, это позволяет ускорить синтез проекта за счет того, что не надо каждый раз при новом синтезе проекта пересинтезировать ядро. Global же генерирует файл описания ядра, используемый каждый раз при новом синтезе проекта для пересинтезирования IP — ядра. Дальнейшие настройки позволяют настроить, на локальной или удаленной машине проводить синтез, либо сгенерировать скрипт для автоматизированного синтеза. Для продолжения требуется нажать *«Generate»*. Сгенерированное IP — ядро должно появиться в окне иерархии проекта во вкладке *«IP Sources»* как показано на рисунке 47.



Рисунок 47 — Список IP — ядер в проекте

Далее можно развернуть IP — ядро как показано на рисунке 48 и посмотреть, что сгенерировала среда, там будут файлы с шаблонами для инстанцирования IP в ваш HDL — код, файлы для симуляции ядра и результаты *«Out-Of-Context»* синтеза в виде *.dcp файла и нетлистов.

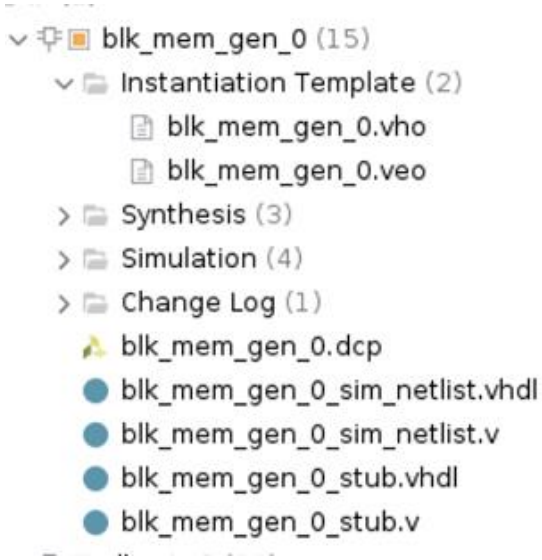


Рисунок 48 — Сгенерированные файлы ядра

5. RTL АНАЛИЗ ПРОЕКТА

В среде разработки Vivado существуют инструменты для анализа вашего RTL — кода на ошибки. При наличии ошибок в коде, информация о них будет отображаться в дереве проекта (рисунок 49а) и в поле *«Messages»* в окне с инструментами анализа, логами, визуализатором репортов (рисунок 49б).

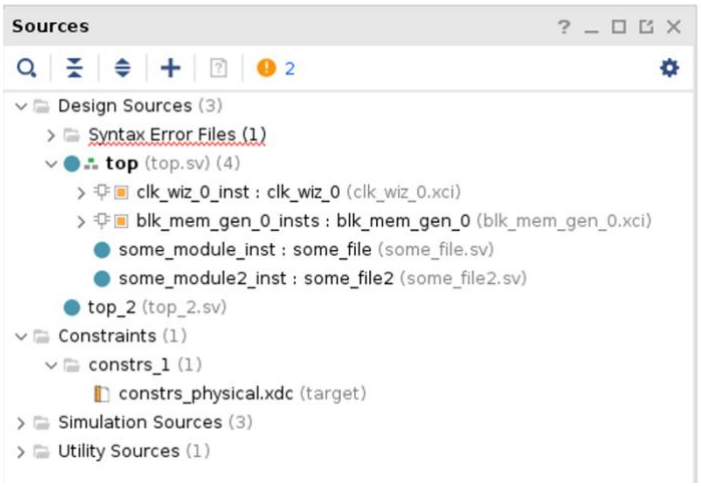


Рисунок 49а — Отображение файлов с ошибками в дереве проекта



Рисунок 49б — Сообщения об ошибках в нижнем окне среды Vivado

Также можно посмотреть в виде схемы, какая логическая схема формируется из вашего RTL — кода. Для этого в окне *«Flow Navigator»* есть пункт *«RTL Analysis»*, развернув меню *«Open Elaborated Design»*, появится кнопка для формирования схемы - *«Schematic»* (Рисунок 50).



Рисунок 50 — Меню пункта *«RTL Analysis»* в окне *«Flow Navigator»*

Будет сгенерирована схема вашего проекта с сохранением иерархии и возможностью развернуть каждый из блоков как показано на рисунке 51.

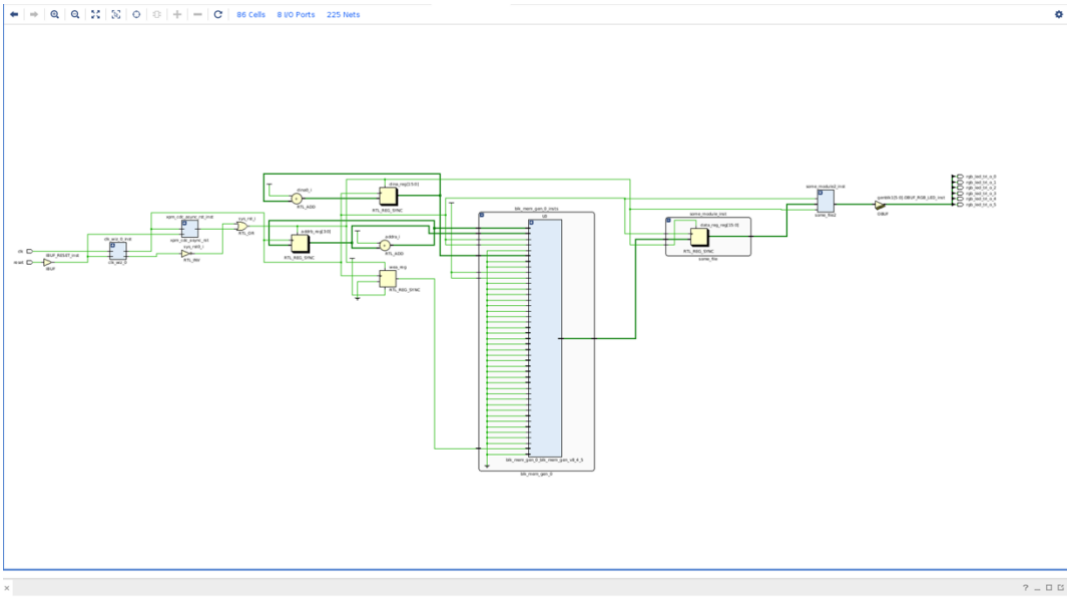


Рисунок 51 — Сгенерированная в *«RTL Analysis»* схема проекта

6. Синтез

Синтез – это процесс преобразования описания дизайна на уровне регистровых передач (RTL) в представление на уровне примитивов ПЛИС. Синтезатор AMD Vivado также производит оптимизации дизайна для уменьшения потребления памяти и повышения производительности получаемой схемы. Vivado поддерживает синтезируемые подмножества следующих стандартов языков:

- SystemVerilog: IEEE Standard for SystemVerilog-Unified Hardware Design Specification, and Verification Language (IEEE Std 1800-2012);
- Verilog: IEEE Standard for Verilog Hardware Description Language (IEEE Std 1364-2005);
- VHDL: IEEE Standard for VHDL Language (IEEE Std 1076-2002);
- VHDL 2008;
- Поддержка одновременного использования в проекте модуле, написанных на разных языках описания аппаратуры.

Для запуска синтеза надо нажать на кнопку *«Run Synthesis»* в меню *«Flow Navigator»* в разделе *«Synthesis»* как показано на рисунке 52. Также в этом меню можно сгенерировать ряд отчетов по результатам синтеза, они станут активны после завершения синтеза.

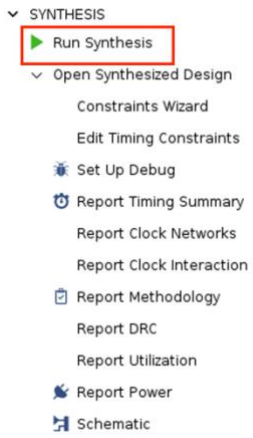


Рисунок 52 — Меню синтеза в окне *«Flow Navigator»*

После нажатия кнопки *«Run Synthesis»* появится меню, в вашем случае при работе на локальном ПК можно оставить настройки по умолчанию, только в вашем случае *«Number of jobs»* будет задано средой автоматически и может быть другим, отличным от того, что на рисунке 53. Нажмите *«OK»* для продолжения.

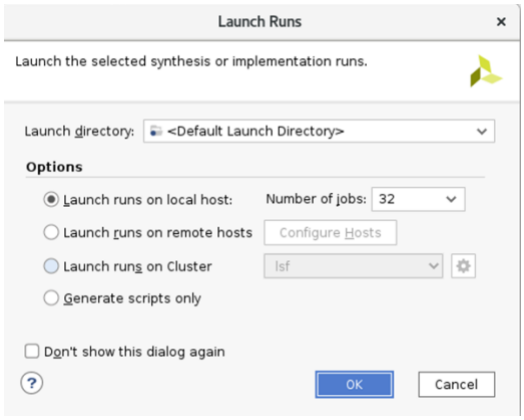


Рисунок 53 — Окно запуска синтеза

В случае если синтез прошел успешно, вы увидите окно как на рисунке 54.

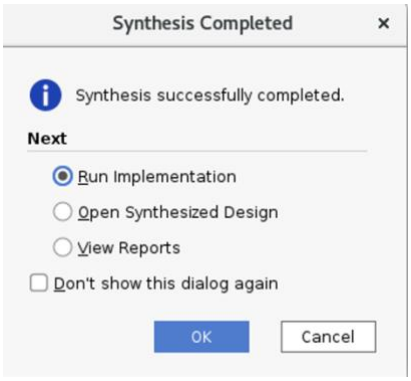


Рисунок 54 — Окно успешного синтеза

В случае наличия ошибок, среда покажет окно как на рисунке 55а, в таком случае вам нужно исправить все ошибки, приведшие к ошибке синтеза, Vivado предложит посмотреть данные об ошибках в разделе *«Messages»* (рисунок 55б).

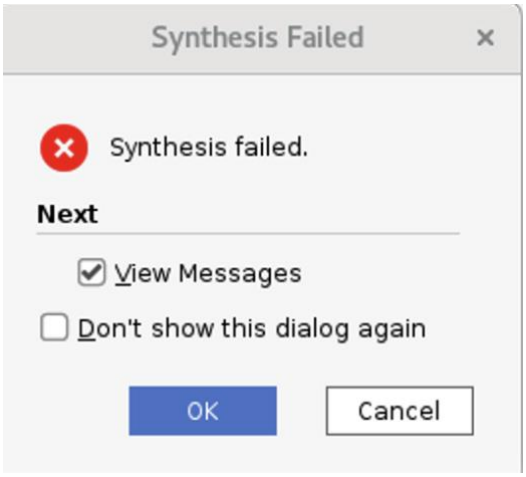


Рисунок 55а — Окно завершения синтеза с ошибками



Рисунок 55б — Окно завершения синтеза с ошибками

Результатом синтеза является набор отчетов о результатах синтеза и файл описания дизайна, который в последствии используется при имплементации.

7. Имплементация

Имплементация в Vivado представляет собой процесс преобразования результатов синтеза и файлов ограничений (констрейнтов) в дизайн, оптимизированный средой и размещенный на выбранную вами ПЛИС. Имплементация состоит из следующих основных этапов:

1. Opt Design: Оптимизация логики для сокращения количества необходимых ресурсов;
2. Place Design: Размещение логических элементов дизайна на ПЛИС с учетом заданных ограничений;
3. Route Design: Трассировка дизайна, то есть соединение размещенных логических элементов.

Для запуска имплементации надо нажать кнопку *«Run Implementation»* в разделе *«Implementation»* в окне *«Flow Navigator»* (рисунок 56).

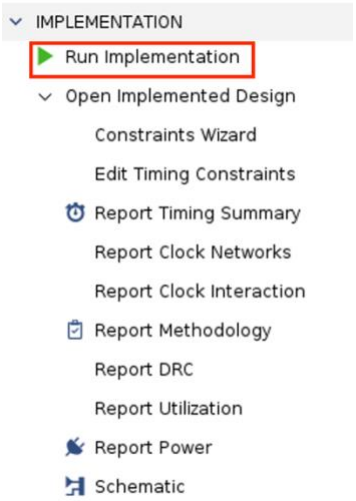


Рисунок 56 — Раздел *«Implementation»* в окне *«Flow Navigator»*

Далее процесс запуска имплементации такой же, как для синтеза. Рекомендуется оставить настройки по умолчанию как на рисунке 57а, только в вашем случае *«Number of jobs»* будет задано средой автоматически и может быть не как на рисунке 57а. На рисунке 57б показано окно в случае успешной имплементации.

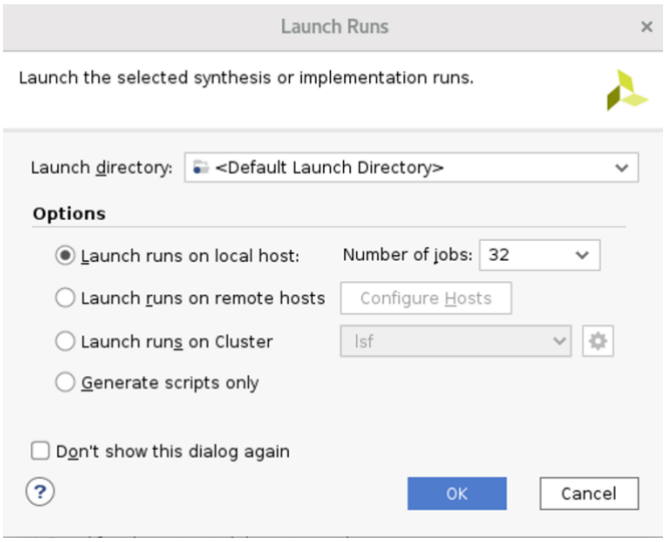


Рисунок 57а — Окно запуска имплементации

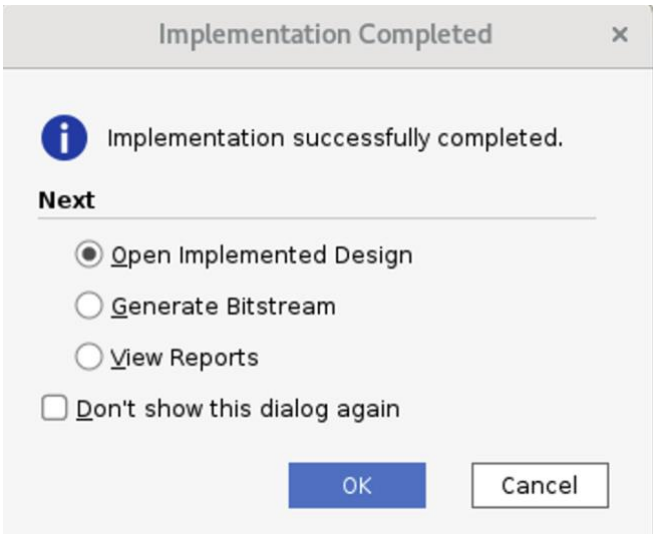


Рисунок 57б — Успешно пройденная имплементация

Vivado позволяет открыть дизайн и посмотреть результаты имплементации в виде схемы, а также посмотреть отчеты по результатам имплементации. Краткую информацию по результатам можно посмотреть в разделе *«Design Runs»* в нижнем окне интерфейса (Рисунок 58). Наибольший интерес представляют результаты статического временного анализа и оценка затраченных ресурсов. Если по какому-либо из параметров будет нарушение, среда подсветит цифру красным цветом. Пример приведен на рисунке 58.

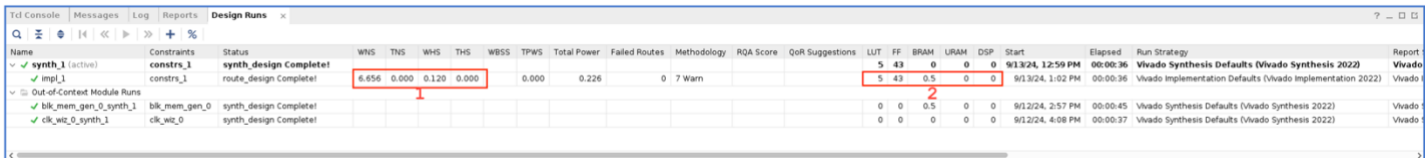


Рисунок 58 — Краткая информация по результатам имплементации

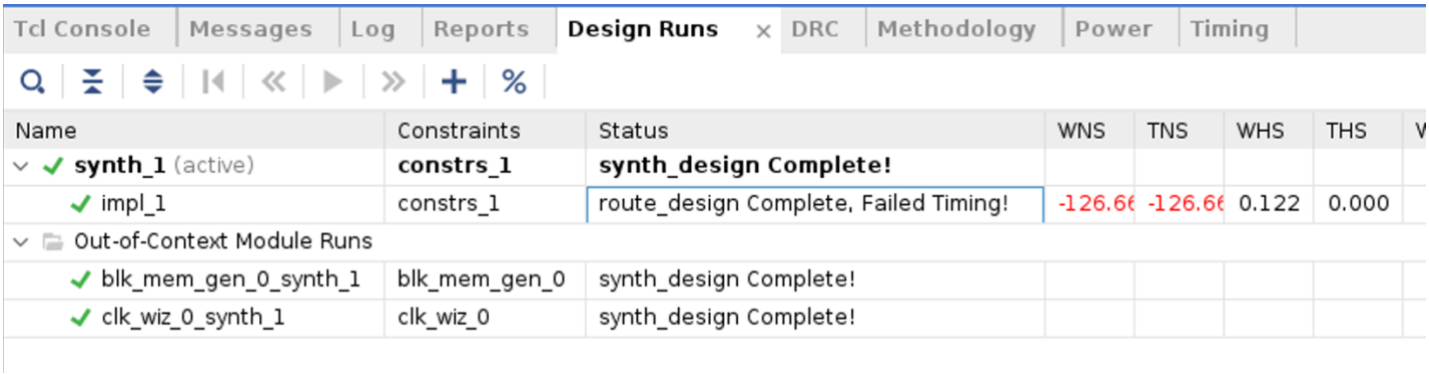


Рисунок 59 — Пример результата имплементации с временными нарушениями

Для того, чтобы посмотреть отчет по временным нарушениям нужно развернуть пункт *«Implementation»* в *«Flow Navigator»*, далее развернуть меню *«Open Implemented Design»* и выбрать пункт *«Report timing summary»* как показано на рисунке 60.

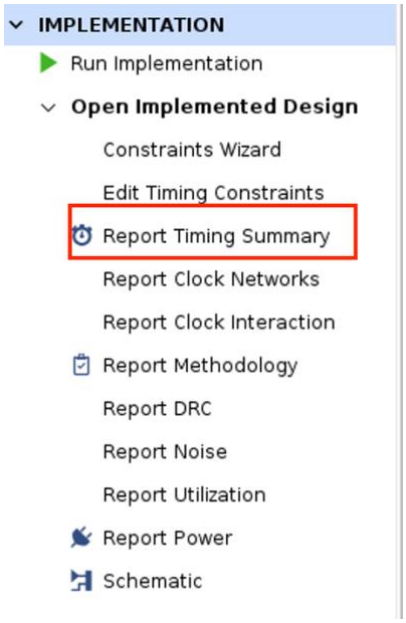


Рисунок 60 — Пункт *«Report timing summary»* в окне *«Flow Navigator»*

Откроется окно настройки отчета, можно оставить настройки по умолчанию(рисунок 61).

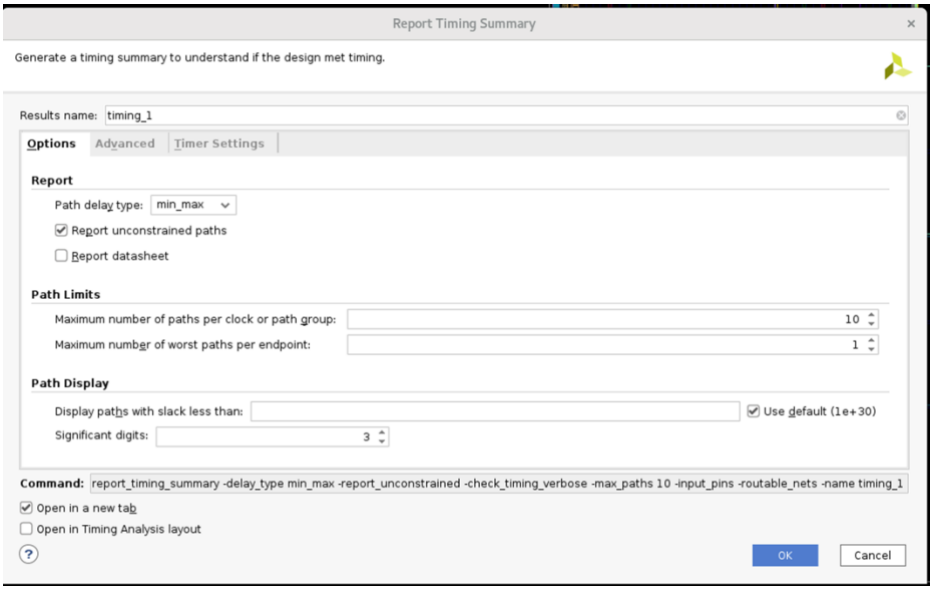


Рисунок 61 — Окно настройки генератора отчета временного анализа

Слева можно развернуть различные категории путей и их принадлежности к тактовым сигналам, красным восклицательным знаком будут подсвечены пункты с нарушениями, развернув их, можно посмотреть, какая цепь вызывает нарушение. В правой области приведен список цепей, относящихся к выбранному тактовому домену (рисунок 62). Данный отчет является результатом статического временного анализа (STA) и позволяет обнаружить и проанализировать критические пути в вашем дизайне. В данном случае для анализа Vivado использует значение частоты тактового сигнала из файлов поддержки платы и заданные в интерфейсе Clocking Wizard частоты тактовых сигналов, так как с IP — ядром генерируются и констрейнты (ограничения) на тактовые сигналы на основе заданных

вами значений. Необходимо исправлять все временные нарушения для корректной работы дизайна. В случае пересечения тактовых доменов, описывать их констрейнтами. А в случае нарушений в одном тактовому домене, оптимизировать критические пути, либо понижать тактовую частоту.

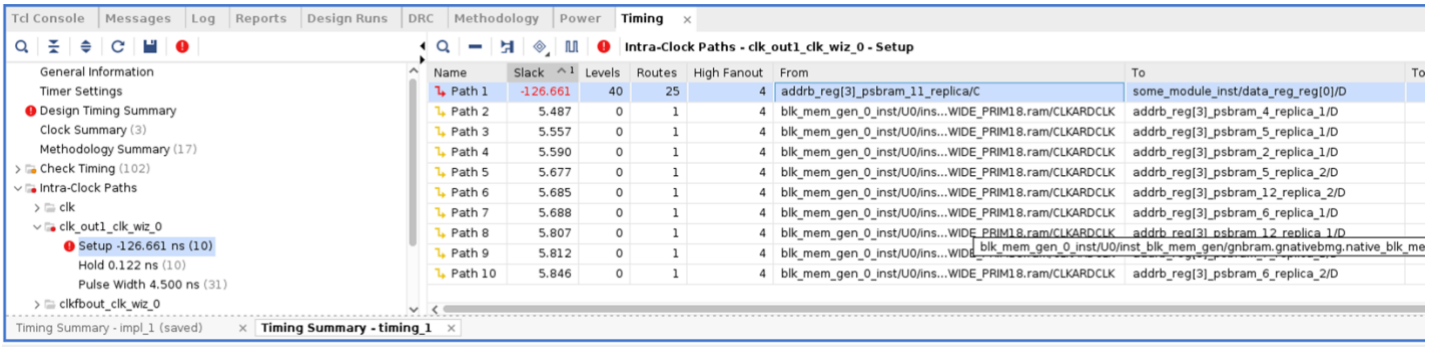


Рисунок 62 — Отчет о временном анализе

В случае больших дизайнов, либо некорректного описания логики, можно столкнуться с ситуацией, когда Vivado не смогла разместить ваш дизайн на кристалле из-за нехватки ресурсов в ПЛИС. Посмотреть, какой модуль ошибочно потребляет много ресурсов либо требует оптимизации по потребляемым ресурсам можно в отчете об утилизации. Его можно открыть в том же меню, где и отчет о временном анализе (рисунок 63а). На рисунке 63б показан интерфейс отчета, можно развернуть модули согласно иерархии проекта.

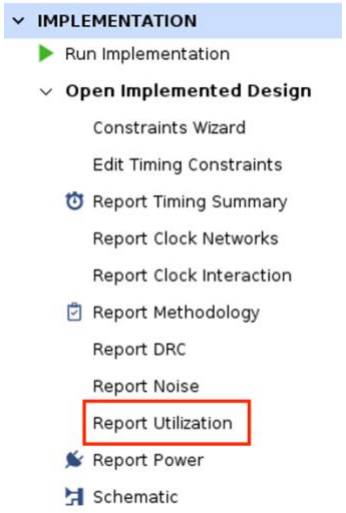


Рисунок 63а — Кнопка для открытия отчета об утилизации ресурсов в пункте *«Implementation»* в окне *«Flow Navigator»*

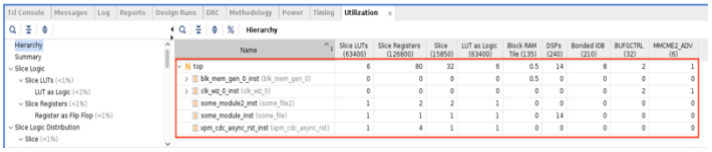


Рисунок 63б — Окно отчета об утилизации ресурсов

Возможно, вам потребуется несколько раз повторять синтез и имплементацию проекта после внесения правок в проект. Когда вы добьетесь отсутствия нарушений в результате имплементации, можно переходить к генерации битстрима.

8. Генерация битстрима и загрузка конфигурации в ПЛИС

Генерация битстрима представляет собой преобразование результатов имплементации в бинарный файл конфигурации ПЛИС. Для генерации битстрима нажмите кнопку *«Generate Bitstream»* в пункте *«Program and debug»* в окне *«Flow Navigator»* как показано на рисунке 64 цифрой 1.



Рисунок 64 — Меню генерации битстрима и подключения к ПЛИС

После завершения генерации можно прошить вашу плату с ПЛИС. Если ваша плата подключена к вашему ПК через USB, нажмите кнопку *«Open Target»*, обозначенный цифрой 2 на рисунке 64. Vivado откроет Hardware Manager и определит ваше устройство. Кнопка *«Program Device»*, отмеченная цифрой 3, должна стать активной. После нажатия на нее начнется процесс загрузки битстрима в ПЛИС.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ И ЛИТЕРАТУРЫ

1. Clocking Wizard LogiCORE IP Product Guide (PG065). — URL: https://docs.amd.com/r/en-US/pg065-clk-wiz/ .
2. Nexys A7™ FPGA Board Reference Manual. — URL: https://digilent.com/reference/_media/reference/programmable-logic/nexys-a7/nexys-a7_rm.pdf .
3. Схема электрическая принципиальная отладочной платы Digilent Nexys A7. — URL: https://digilent.com/reference/_media/programmable-logic/nexys-a7/nexys-a7-d3-sch.pdf .
4. DSC1033 Low-Power CMOS Oscillator Data Sheet. — URL: https://ww1.microchip.com/downloads/en/DeviceDoc/DSC1033%20Datasheet%20MKQBPD0318091-7.pdf .
5. Block Memory Generator v8.4 Product Guide (PG058). — URL: https://docs.amd.com/v/u/en-US/pg058-blk-mem-gen .